Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Social Network Analysis Labs in R and SoNIA

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents

McFarland, Daniel A., Solomon Messing, Michael Nowak and Sean J. Westwood.

To run the following labs install R (Linux, MacOS X or Windows) and execute the following command in R (this will download and install all needed packages and data):

```
source("http://sna.stanford.edu/setup.R")
```

Chapters

1. "Introductory Lab." Nowak, Michael and Daniel A. McFarland. 2010.

2. "Methodological Beginnings – Basic Triadic and Cohesion Measures." Nowak, Michael and Daniel A. McFarland. 2010.

3. "Clusters, Factions and Cores." Nowak, Michael and Daniel A. McFarland.

4. "Centralities and Their Interrelation." Sukumaran, Abhay,Michael Nowak and Daniel A. McFarland.

5. "Affiliation Data and Network Mobility." Messing, Solomon and Daniel A. McFarland. 2010.

6. "Structural Equivalences and Block-Modeling." Nowak, Michael, Solomon Messing, Sean J. Westwood and Daniel A. McFarland. 2010.

7. "Peer Influence and QAP Regression." Messing, Solomon, Sean J. Westwood and Daniel A. McFarland. 2010.

8. "Exponential-Family Random Graph Models." Westwood, Sean J. and Daniel A. McFarland. 2010.

9. "Converting igraph to SoNIA with R." Westwood, Sean J. and Daniel A. McFarland. 2010.

10. "rSoNIA and Visualizing Social Network Dynamics." Bender-deMoll, Skye and Daniel A. McFarland. 2010.

Additional software

SoNIA is a Java-based package for visualizing dynamic or longitudinal "network" data. (This is a temporary download meant to fix SoNIA while a new release is under development.)

Software required to run SoNIA:

- Java 1.6
- Flash player

Acknowledgements: Special thanks to Skye Bender-deMoll and James Moody. An earlier version of the introductory lab, the Exponential Random Graph Model lab, and rSoNIA lab were developed in collaboration with them. We have revised, extended and reorganized the content of those labs here.

This material is based upon work supported by the Office of the President at Stanford University and the National Science Foundation under Grants No. 0835614 and 0624134. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of Stanford University or the National Science Foundation.

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 1 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- 1.1 Krackhardt Full.pdf
- 1.2 Krackhardt Advice.pdf
- 1.3 Krackhardt Friendship.pdf
- 1.4 Krackhardt Reports.pdf
- 1.5 Krackhardt Reports Fruchterman Reingold.pdf
- 1.6 Krackhardt Reports Color.pdf
- 1.7 Krackhardt Reports Vertex Size.pdf
- 1.8 Krackhardt Overlayed Ties.pdf
- 1.9 Krackhardt Overlayed Structure.pdf

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 2 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- console output.txt
- krack node stats.csv
- krack triads.csv

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 3 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- Rplot001.png
- Rplot002.png
- Rplot003.png
- Rplot004.png
- Rplot005.png
- Rplot006.png
- Rplot007.png
- Rplot008.png
- Rplot009.png
- Rplot010.png
- Rplot011.png
- Rplot012.png
- Rplot013.png
- Rplot014.png
- Rplot015.png
- Rplot016.png
- Rplot017.png
- Rplot018.png
- Rplot019.png
- Rplot020.png
- console output.txt

- movie.gif

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 4 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- console output.txt

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 5 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- 5.1 magact stdnt actvts 1996.pdf
- 5.2 magact stdnt actvts 1996 layout.kamada.kawai.pdf
- 5.3 magact stdnt actvts 1996 layout.fruchterman.reingold.grid.pdf
- 5.4 magact stdnt actvts 1996 layout.fruchterman.reingold.pdf
- 5.5 magact stdnt actvts 1997.pdf
- 5.6 magact stdnt actvts 1998.pdf
- 5.7 magact stdnt actvts 1996 clubs.pdf
- 5.8 magact stdnt actvts club overlap density.pdf
- 5.9 magact stdnt actvts club overlap handplaced layout.pdf
- console output.txt

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 6 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- 6.1 m182 studentnet friend social task plots.pdf
- 6.2 m182 studentnet social hclust.pdf
- 6.3 m182 studentnet task social clustered observed corrs.pdf
- 6.4 m182 studentnet task triad hclust.pdf
- 6.5 m182 studentnet task hclust triad corrs.pdf
- 6.6 m182 studentnet task social pca scree.pdf
- 6.7 m182 studentnet task social pca hclust.pdf
- 6.8 m182 task cluster by correlation PCA Triads.pdf
- console output.txt

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 7 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- 7.1 Peer effect on opinion of subject.pdf
- 7.2 dotplot matchoutput bootstrapped SE.pdf
- console output.txt

- Stanford University

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 8 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- 8.1 lab8 network.pdf
- 8.2 lab8 mcmc m1.pdf
- 8.3 lab8 mcmc m2.pdf
- 8.4 lab8 m2 simulation.pdf
- 8.5 lab8 m2 gof.pdf
- console output.txt

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Lab 9 Source

Citation

McFarland, Daniel, Solomon Messing, Michael Nowak, and Sean J. Westwood. 2010. "Social Network Analysis Labs in R." Stanford University.

Home » Lab table of contents » Lab details

Download source R file | Download lab reading list

Output files

- console output.txt
- export.son
- network.swf

SoNIA - Social Network Image Animator

Introductory Lab for SoNIA

Aim: to provide a brief practical introduction to SoNIA and some conceptual background on dynamic network visualization. This document was written by Dan McFarland for use in demonstrating SoNIA in his classrooms.

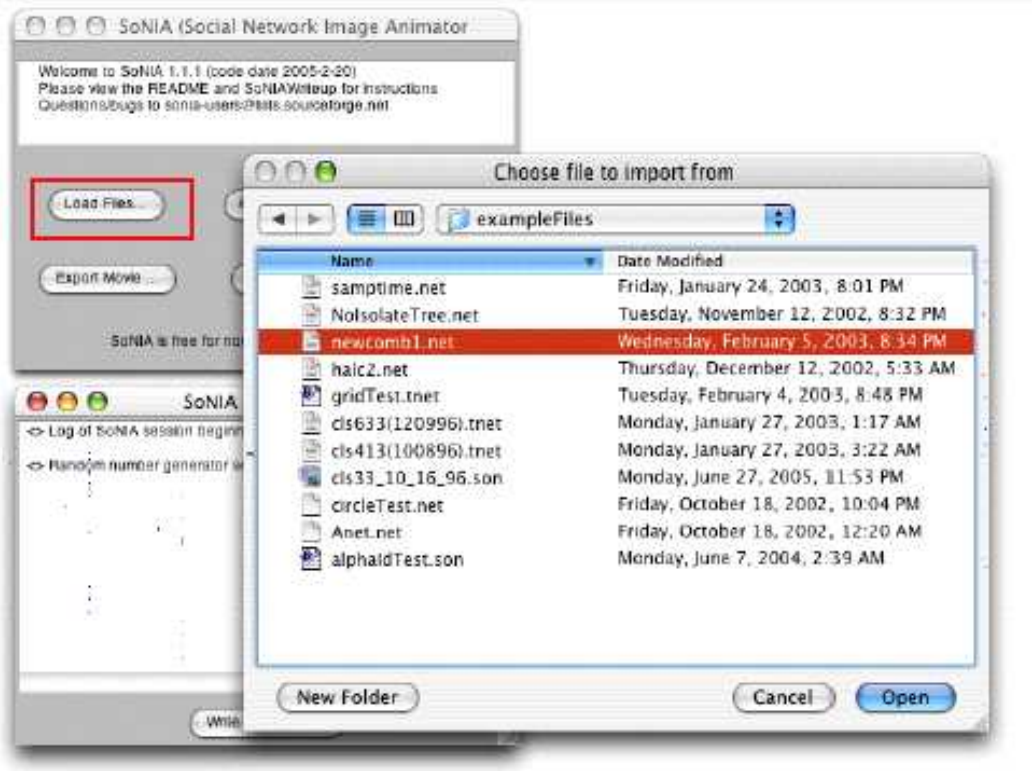
For full functionality, SoNIA requires at least Java 1.3.1, QuickTime, and QTJava be installed on your computer. To download SoNIA, and for full details and installation instructions, please visit http://sonia.stanford.edu/documentation/install.html

1. Starting SoNIA

SoNIA can be launched by double clicking on the sonia.jar file, but will not provide full information about errors. It is better to launch SoNIA directly by using the java command in the terminal window (the dos "C/" prompt on windows machines). Navigate to the directory containing SoNIA and type:

```
java -cp sonia.jar sonia.SoniaController
```

2. Loading the Newcomb data



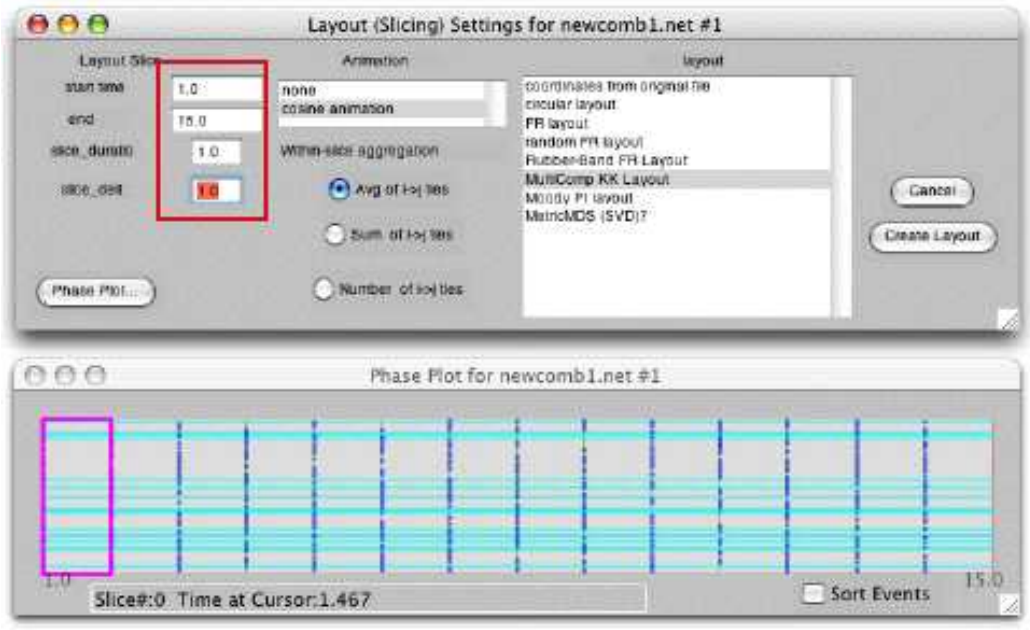
Click the "Load Files..." button. Navigate to the "necomb1.net" file and open it. Click OK in the window that pops up. If the file loads successfully, some information about the file will be printed in the "SoNIA Session Log" window, otherwise an error message will appear details

the problem.

The log tells how many node and arc events (how many individual records there are in the data, not the number present at one time). It also displays the smallest and largest time values in the data, this gives overall time range you may want to examine. The Newcomb data consists of an acquaintance network sampled at 15 time points, so the data are in discrete waves.

3. Creating a Layout (for Newcomb)

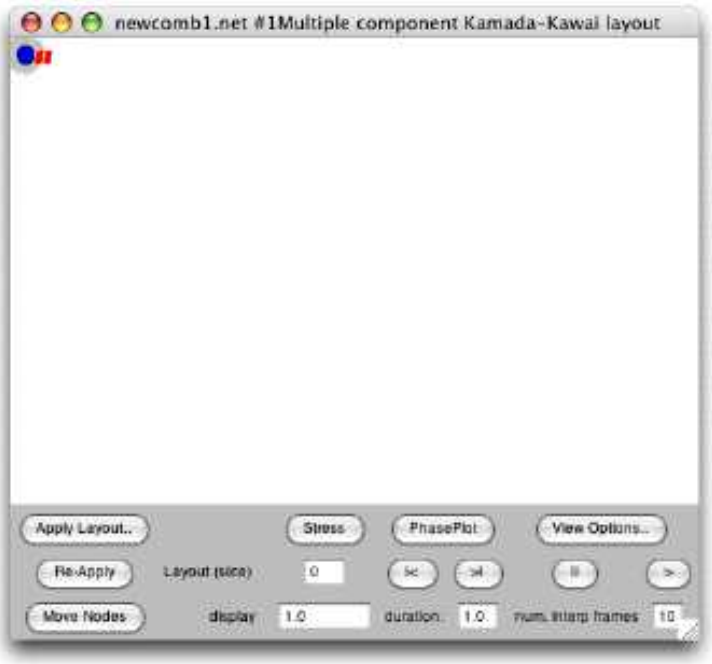
Click the "Create Layout..." button. This will bring up a window allowing to make decisions about what range of the data to work with, the layout technique, etc. "Create Layout" always applies to the most recently loaded network, but multiple networks can be loaded, and/or multiple layouts for the same network created. "Start Time" and "End Time" give the temporal range for the layout. Enter 1 and 15. "Slice duration" gives the length of time to be used for each bin when constructing the "slice" networks.



The "phase plot" button brings up a window which shows a time map of the data you are working with. Dark blue indicates arc events, light blue indicates nodes, and the length of lines

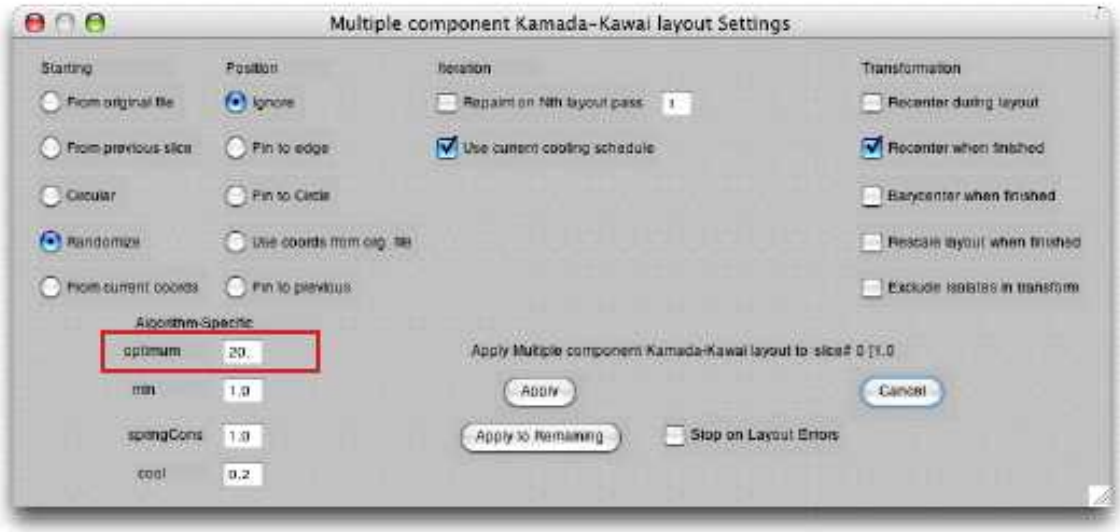
indicates their position in time. The purple square is a preview of a slice. The window will update when you return while editing one of the layout slice fields.

Because Newcomb is discrete "integer time" data, it is "binned" when the data was collected. If you want each frame to correspond to the original waves, you will probably want to enter 1. "Slice Delta" controlled the offset between successive slices, set this to the same as the slice size (1) , for non-overlapping slices (more on this later). Set the "Slice aggregation settings to "Avg. of i->j ties." Set the layout type to "Multi Component Kamada-Kawai". Click "Create Layout". A layout window should appear. All the nodes will be drawn in the top left because no layout has been applied.

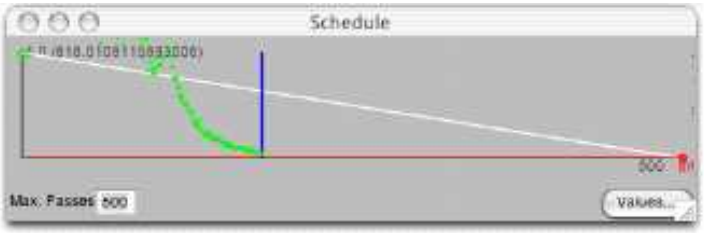


4. Applying a Layout algorithm (MultiComponent Kamada-Kawai)

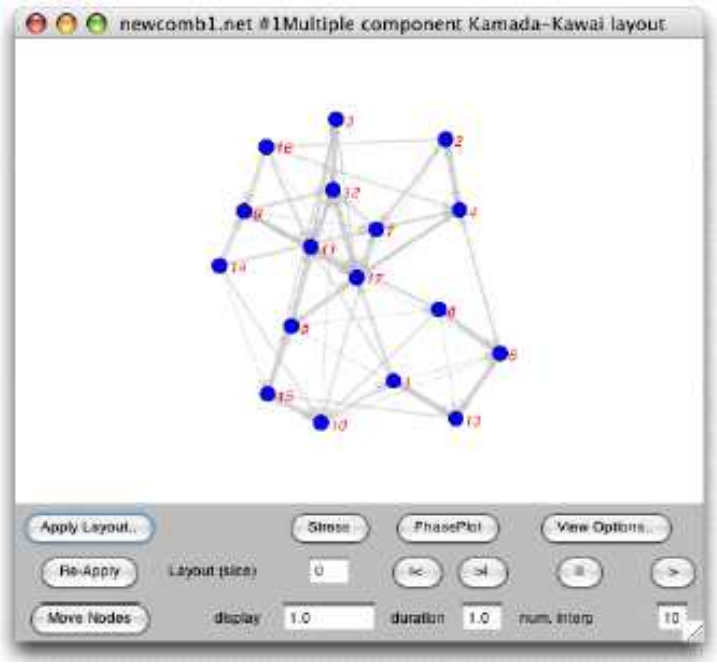
Click "Apply Layout.." to bring up the layout settings dialog. Set the starting coordinates to "Randomize" or "Circular". Click Apply. The Kamada-Kawai "spring embedder" layout will be applied to the first slice (slice 0).



The green dots on the "Schedule" window give information on the algorithms convergence (the change in the "energy" of the layout after each iteration of the algorithm.) In some rare cases, the layout will fail to converge, this will show as a cycle (parallel lines) in the "schedule plot". Or it may need more iterations to arrive at a solution, in which case set the value of "Max. Passes" higher.



The layout drawn in the window will be fairly "crunched." It doesn't take advantage of much of the screen space, and it is hard to see the ties. The "optimal distance" between nodes that the layout is aiming for can be adjusted. Click on "Apply Layout" again. Enter 50 in the "Opt. Dist." field, and click "apply." The layout should be much larger. To adjust the display further, click on "View Options" to bring up the graphic settings dialog. Un-check "Transparency", and set Node and Arc scale factors to 0.5.

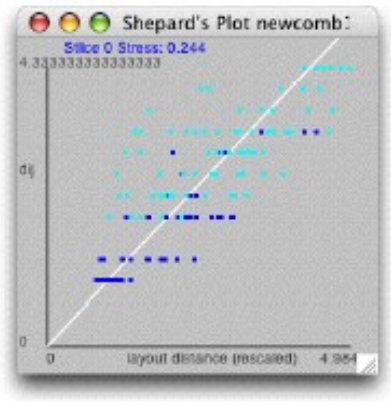


At this point, a layout has only been created for the first "slice" network. To create the remaining networks, enter 1 in the "Layout slice" field and hit return, or click the ">|" button to advance to the next slice. All the nodes will appear in the upper left corner, at the origin of the coordinate axes. At this point you could click "Re-Apply" to apply the same layout settings to this slice. However, KK usually gives the best results when each layout is started from the coordinates of the previous layout, as this helps the algorithm find an optimia that is close in coordinate space to the previous layout. Click "Apply Layout", and set the starting coordinates to "From Previous Slice", then click "Apply to Remaining" to apply these layout settings to all the rest of the slices. When it has finished, use the "|<" and ">|" buttons to step through the "movie" one slice at a time, or go back to the beginning and press ">" to watch the whole thing. (you can also navigate by bringing up the phase plot window and clicking on slices) To make the animation smoother/slower, enter a larger number in the "interp. frames field" - this controls how many frames will be rendered between the slices (you will need to set the "duration" field back to 1).

5. Evaluating the "Stress" of the layout

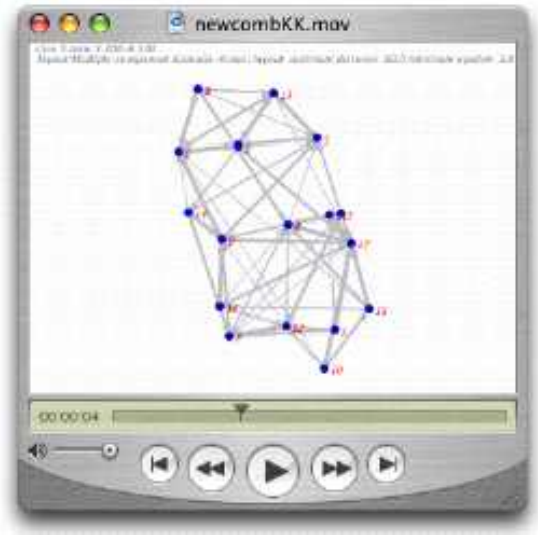
Most social networks require a high dimensional layout in order to accurately represent the relative distances. Projecting down to the two dimensions of the screen usually results in some degree of distortion. To get a sense of how distorted a given layout is, click the "Stress.." button. This brings up a "Shepard's Stress Plot" window. The desired distances for each arc in the network matrix (in the case of KK, the distance matrix is created by converting the adjacency values to dissimilarities, and then finding the all-pairs-shortest-path or "Geodesic distance" between each of the nodes) are plotted against the arc's length on the screen layout. In a hypothetical zero-distortion layout, all of the points would lie on the white diagonal, so

the scatter gives a qualitative measure of the distortion. The dark blue points indicate the direct ties (the arcs actually drawn on screen) and the light blue points indicate the multiple step relations. The "stress" value given is an adaptation of "Kruskal's Stress" from MDS. Values below 0.2 are generally considered to be "acceptable".



To see how node positions effect the stress, click the "move nodes" button in the layout to turn on the manual adjustment mode. Drag one of the nodes to a new position on the screen, and recalculate the stress. Be sure to click the "Stop Moving" button to end the adjustment mode when you are finished.

6. Saving the network animation as a QuickTime movie



When an animation is viewed in SoNIA, the rate of movement is controlled by the speed at which Java can draw each of the frames to the screen. So networks with many arcs and edges will take longer, and appear to move more slowly. Also, it is often useful to store the results of a layout, show it over the web, etc. To export an animation as a QuickTime movie, click the "Export Movie..." button in the main controller window. Choose the layout to export (if you have created more than one). Chose a name and location for the movie file (file name should end in .mov). Click OK. A window will popup showing the movie export frame by frame as it saves out. When it has finished, the movie can be played in the QuickTime MoviePlayer, or in

most web browsers. Because the movie is literally a sequence of pictures of the network, the file will be quite large, and it is not possible to further edit the network. There will be some loss of image quality due to compression, we hope to eventually use a better compressor.

8. Saving out the log file

The log file records (most of) the parameters, settings, and the sequence of actions used to create your layout. Generally it is a good idea to save out the log file at the same time you save the movie. That way you will have enough information to reconstruct the movie at a later date, and will be able to remember the exact details. Click the "Write Log to File" button in the Log window, choose a name and location for the file.

----- For more information on how to use additional layout algorithms, configuring data for input, etc. please check the website at http://sonia.stanford.edu/ or email skyebend@stanford.edu.

```
##########################################################################
# You may cite these labs as follows: McFarland, Daniel, Solomon Messing,
# Mike Nowak, and Sean Westwood. 2010. "Social Network Analysis
# Labs in R." Stanford University.
##########################################################################


##########################################################################
# LAB 1 - Introductory Lab
# The point of this lab is to introduce students to the packages of
# SNA and Igraph, to cover some basic R commands, to load and manage
# data, to generate graph visualizations, and to export the data for
# use elsewhere.
##########################################################################

###
# 0. R BASICS
###

# Any line starting with # is a "comment" line and is ignored by
# R. Any other line is treated as a command. Run commands by
# copying and pasting them into the R Console.
#
# If (when) you get confused, a good place to start is with R's
# built-in help functionality. R offers detailed help files for
# each function and each package. To access help type ?[function
# or package name] in the console. For example, for help on the
# "sum" function, type:
?sum

# To install all packages need for Social Network Analysis
# Labs in R, uncomment and run the following code:

#source("http://sna.stanford.edu/setup.R")

# You only need to run this once per computer!

# To load the packages from , you need to call the "library"
# command. Note that you need to do this each session; packages
# don't load automatically by default (though you can set this
# as a preference if you'd like).
```

```
# For this lab, we will use the "igraph" package.
# A manual is available at
# http://cran.r-project.org/web/packages/igraph/igraph.pdf.
library(igraph)

# Sometimes, different packages overlap in functionality and
# cause unexpected behavior when both are loaded simultaneously.
# If you ever want to remove an existing library, use the
# "detach" command:
#
# detach(package:igraph)

# IMPORTANT NOTE: Unlike in most languages, R objects are numbered
# from 1 instead of 0, so if you want the first element in a
# vector, you would reference it by vector_name[1]. HOWEVER,
# igraph objects are numbered starting from 0. This can lead to
# lots of confusion, since it's not always obvious at first which
# objects are native to R and which belong to igraph.



###
# 1. LOADING DATA
###

# The <- operator sets a variable equal to something. In this case,
# we will set a number of basic R data structures, called "data
# frames," to hold the contents of the files we will open.
#
# read.table() is the most common R command for loading data from
# files in which values are in tabular format. The function loads
# the table into a data frame object, which is the basic data type
# for most operations in R. By default, R assumes that the table
# has no header and is delimited by any white space; these
# settings are fine for our purposes here.
#
# One handy aspect of R is that you can read in data from a URL
# directly by referencing the URL in the read.table() function,
# as follows:
advice_data_frame <- read.table('http://sna.stanford.edu/sna_R_labs/data/Krack-High-Tec-edgelist-
Advice.txt')
friendship_data_frame <- read.table('http://sna.stanford.edu/sna_R_labs/data/Krack-High-Tec-edgelist-
Friendship.txt')
reports_to_data_frame <- read.table('http://sna.stanford.edu/sna_R_labs/data/Krack-High-Tec-edgelist-
```

```
ReportsTo.txt')

# If the files you want to work with are on your local machine,
# the easiest way to access them is to first set your working
# directory via the setwd() command, and then reference the
# files by name:
#
# setwd('path/to/your_directory')
# your_data_frame <- read.table('your_file_name')

# Note that when you set a variable equal to something, if all
# goes well R will not provide any feedback. To see the data we
# just loaded, it's necessary to call the variables directly.
advice_data_frame

# Since this is a bit long, we can see just the top six rows via
# head()...
head(friendship_data_frame)

# ... or the bottom six rows via tail().
tail(reports_to_data_frame)

# To view your data in a spreadsheet-like window, use the command 'fix()'.
fix(reports_to_data_frame)

# The attribute data for this lab is in a comma-separated-value
# (CSV) file. read.csv() loads a CSV file into a data frame
# object. In this case, we do have a header row, so we set
# header=T, which tells R that the first row of data contains
# column names.
attributes <- read.csv('http://sna.stanford.edu/sna_R_labs/data/Krack-High-Tec-Attributes.csv',
header=T)
attributes

# Other commands may be used to load data from files in different
# formats. read.delim() is a general function for loading any
# delimited text file. The default is tab-delimited, but this can
# be overridden by setting the "sep" parameter. For example:
#
#     f <- read.delim("tab_delimited_file.txt")
#     f <- read.delim("colon_delimited_file.txt", sep=':')
#
# The 'foreign' package will allow you to read a few other
```

```
# custom data types, such as SPSS files via read.spss() and
# STATA files via read.dta().

# When data files are part of an R package you can read them as
# follows:
#
# data(kracknets, package = "NetData")

# In the future, we will load data this way. However, it is useful
# to get a sense of how things often must be done in R.


###
# 2.2. LOADING GRAPHS
###

# For convenience, we can assign column names to our newly
# imported data frames. c() is a common generic R function that
# combines its arguments into a single vector.
colnames(advice_data_frame) <- c('ego', 'alter', 'advice_tie')
head(advice_data_frame)

colnames(friendship_data_frame) <- c('ego', 'alter', 'friendship_tie')
head(friendship_data_frame)

colnames(reports_to_data_frame) <- c('ego', 'alter', 'reports_to_tie')
head(reports_to_data_frame)

# Take a look at each data frame using the 'fix()" function. Note that you'll
# need to close each fix window before R will evaluate the next line of code.
fix(advice_data_frame)
fix(friendship_data_frame)
fix(reports_to_data_frame)

# Before we merge these data, we need to make sure 'ego' and 'alter' are the
# same across data sets. We can compare each row using the == syntax.
# The command below should return TRUE for every row if all ego rows
# are the same for advice and friendship:
advice_data_frame$ego == friendship_data_frame$ego

# That's a lot of output to sort through. Instead, we can just have R return
# which row entries are not equal using the syntax below:
which(advice_data_frame$ego != friendship_data_frame$ego)
```

```
# Repeat for other variables
which(advice_data_frame$alter != friendship_data_frame$alter)
which(reports_to_data_frame$alter != friendship_data_frame$alter)
which(reports_to_data_frame$ego != friendship_data_frame$ego)

# Now that we've verified they are all the same, we can combine them into
# a single data frame.
krack_full_data_frame <- cbind(advice_data_frame,
      friendship_data_frame$friendship_tie,
      reports_to_data_frame$reports_to_tie)
head(krack_full_data_frame)

# Notice that the last two variable names are now
# "reports_to_data_frame$reports_to_tie"
# and "friendship_data_frame$friendship_tie".
# That's a little long. We can rename them
# as follows:

names(krack_full_data_frame)[4:5] <- c("friendship_tie",
      "reports_to_tie")
head(krack_full_data_frame)

# Another way to build the data frame is to use R's
# data.frame syntax from the start:
krack_full_data_frame <- data.frame(ego = advice_data_frame[,1],
      alter = advice_data_frame[,2],
      advice_tie = advice_data_frame[,3],
      friendship_tie = friendship_data_frame[,3],
      reports_to_tie = reports_to_data_frame[,3])
head(krack_full_data_frame)



# Now let's move on to some data processing.

# Reduce to non-zero edges so that the edge list only contains
# actual ties of some type.
krack_full_nonzero_edges <- subset(krack_full_data_frame,
      (advice_tie > 0 | friendship_tie > 0 | reports_to_tie > 0))
head(krack_full_nonzero_edges)

# Now we can import our data into a "graph" object using igraph's
# graph.data.frame() function. Coercing the data into a graph
```

```
# object is what allows us to perform network-analysis techniques.
krack_full <- graph.data.frame(krack_full_nonzero_edges)
summary(krack_full)

# By default, graph.data.frame() treats the first two columns of
# a data frame as an edge list and any remaining columns as
# edge attributes. Thus, the 232 edges appearing in the summary()
# output refer to the 232 pairs of vertices that are joined by
# *any type* of tie. The tie types themselves are listed as edge
# attributes.

# To get a vector of edges for a specific type of tie, use the
# get.edge.attribute() function.
get.edge.attribute(krack_full, 'advice_tie')
get.edge.attribute(krack_full, 'friendship_tie')
get.edge.attribute(krack_full, 'reports_to_tie')

# If you would like to symmetrize the network, making all
# asymmetric ties symmetric, use the as.undirected()
# function:
krack_full_symmetrized <- as.undirected(krack_full, mode='collapse')
summary(krack_full_symmetrized)




###
# 3. ADDING VERTEX ATTRIBUTES TO A GRAPH OBJECT
###

# One way to add the attributes to your graph object is to iterate
# through each attribute and each vertex. This means that we will
# add one attribute at a time to each vertex in the network.
#
# V(krack_full) returns a list of the IDs of each vertex in the
# graph. names(attributes) returns a list of the column names in
# the attributes table. The double-for loop tells R to repeat the
# code between the brackets once for each attribute and once for
# each vertex.
for (i in V(krack_full)) {
   for (j in names(attributes)) {
      krack_full <- set.vertex.attribute(krack_full,
                                         j,
                                         index = i,
```

```
                                attributes[i + 1, j])
    }
}

# A shorter way is to just read in attribute names when you
# create the graph object:

# First create a vector of vertex labels, in this case 1:n
attributes = cbind(1:length(attributes[,1]), attributes)

krack_full <- graph.data.frame(d = krack_full_nonzero_edges,
                               vertices = attributes)

# Note that we now have 'AGE,' 'TENURE,' 'LEVEL,' and 'DEPT'
# listed alongside 'name' as vertex attributes.
summary(krack_full)

# We can see a list of the values for a given attribute for all of
# the actors in the network.
get.vertex.attribute(krack_full, 'AGE')
get.vertex.attribute(krack_full, 'TENURE')
get.vertex.attribute(krack_full, 'LEVEL')
get.vertex.attribute(krack_full, 'DEPT')



###
# 4. VISUALIZE THE NETWORKS
###

# We can use R's general-purpose plot() method to generate custom
# visualizations of the network.

# R only lets us look at one plot at a time.  To make our work easier
# we will save our plots as PDF files.  To jus create a plot execute
# the code between the PDF function and "dev.off()".

# In order to save PDF files we must tell R where to put them.  We do
# this with the setwd() command.  You must put the full path to the
# folder where you will output the files here.

# In OS X you can get this information by selecting the folder, right
# clicking and selecting "Get Info."  The path is listed under "Where."
```

```
# In Windows you can get this information by selecting the folder, right
# clicking and selecting "Properties."  The path information is listed
# "location".

# example: setwd("/Users/seanwestwood/Desktop/lab_1")
setwd("")

# First, let's plot the network with all possible ties.
pdf("1.1_Krackhardt_Full.pdf")
plot(krack_full)
dev.off()

# This is a bit of a jumble, so let's look at the networks for
# single edge types.

# advice only
krack_advice_only <- delete.edges(krack_full,
   E(krack_full)[get.edge.attribute(krack_full,
   name = "advice_tie") == 0])
summary(krack_advice_only)
pdf("1.2_Krackhardt_Advice.pdf")
plot(krack_advice_only)
dev.off()

# friendship only
krack_friendship_only <- delete.edges(krack_full,
   E(krack_full)[get.edge.attribute(krack_full,
   name = "friendship_tie") == 0])
summary(krack_friendship_only)
pdf("1.3_Krackhardt_Friendship.pdf")
plot(krack_friendship_only)
dev.off()

# reports-to only
krack_reports_to_only <- delete.edges(krack_full,
   E(krack_full)[get.edge.attribute(krack_full,
   name = "reports_to_tie") == 0])
summary(krack_reports_to_only)
pdf("1.4_Krackhardt_Reports.pdf")
plot(krack_reports_to_only)
dev.off()

# Still kind of messy, so let's clean things up a bit. For
```

```
# simplicity, we'll focus on reports_to ties for now.

# First, we can optimize the layout by applying the layout
# algorithm to the specific set of ties we care about. Here
# we'll use Fruchterman-Rheingold; other options are
# described in the igraph help page for "layout," which
# can be accessed by entering ?layout.

reports_to_layout <- layout.fruchterman.reingold(krack_reports_to_only)
pdf("1.5_Krackhardt_Reports_Fruchterman_Reingold.pdf")
plot(krack_reports_to_only,
    layout=reports_to_layout)
dev.off()

# Now let's color-code vertices by department and clean up the
# plot by removing vertex labels and shrinking the arrow size.
dept_vertex_colors = get.vertex.attribute(krack_full,"DEPT")
colors = c('Black', 'Red', 'Blue', 'Yellow', 'Green')
dept_vertex_colors[dept_vertex_colors == 0] = colors[1]
dept_vertex_colors[dept_vertex_colors == 1] = colors[2]
dept_vertex_colors[dept_vertex_colors == 2] = colors[3]
dept_vertex_colors[dept_vertex_colors == 3] = colors[4]
dept_vertex_colors[dept_vertex_colors == 4] = colors[5]

pdf("1.6_Krackhardt_Reports_Color.pdf")
plot(krack_reports_to_only,
   layout=reports_to_layout,
   vertex.color=dept_vertex_colors,
   vertex.label=NA,
   edge.arrow.size=.5)
dev.off()
# Now let's set the vertex size by tenure.
tenure_vertex_sizes = get.vertex.attribute(krack_full,"TENURE")

pdf("1.7_Krackhardt_Reports_Vertex_Size.pdf")
plot(krack_reports_to_only,
    layout=reports_to_layout,
    vertex.color=dept_vertex_colors,
    vertex.label=NA,
    edge.arrow.size=.5,
    vertex.size=tenure_vertex_sizes)
dev.off()
```

```
# Now let's incorporate additional tie types. We'll use the
# layout generated by the reports-to ties but overlay the
# advice and friendship ties in red and blue.

tie_type_colors = c(rgb(1,0,0,.5), rgb(0,0,1,.5), rgb(0,0,0,.5))
E(krack_full)$color[ E(krack_full)$advice_tie==1 ] = tie_type_colors[1]
E(krack_full)$color[ E(krack_full)$friendship_tie==1 ] = tie_type_colors[2]
E(krack_full)$color[ E(krack_full)$reports_to_tie==1 ] = tie_type_colors[3]
E(krack_full)$arrow.size=.5
V(krack_full)$color = dept_vertex_colors
V(krack_full)$frame = dept_vertex_colors

pdf("1.8_Krackhardt_Overlayed_Ties.pdf")
plot(krack_full,
    layout=reports_to_layout,
    vertex.color=dept_vertex_colors,
    vertex.label=NA,
    edge.arrow.size=.5,
    vertex.size=tenure_vertex_sizes)



# Add a legend. Note that the plot window must be open for this to
# work.
legend(1,
      1.25,
      legend = c('Advice',
                 'Friendship',
                 'Reports To'),
      col = tie_type_colors,
      lty=1,
      cex = .7)
dev.off()

# Another option for visualizing different network ties relative
# to one another is to overlay the edges from one tie type on the
# structure generated by another tie type. Here we can use the
# reports-to layout but show the friendship ties:

pdf("1.9_Krackhardt_Overlayed_Structure.pdf")
plot(krack_friendship_only,
    layout=reports_to_layout,
    vertex.color=dept_vertex_colors,
    vertex.label=NA,
```

```
    edge.arrow.size=.5,
    vertex.size=tenure_vertex_sizes,
    main='Krackhardt High-Tech Managers')
dev.off()


###
# 5. EXPORT THE NETWORK
###

# The write.graph() function exports a graph object in various
# formats readable by other programs. There is no explicit
# option for a UCINET data type, but you can export the graph
# as a Pajek object by setting the 'format' parameter to 'pajek.'
# Note that the file will appear in whichever directory is set
# as the default in R's preferences, unless you previously
# changed this via setwd().
write.graph(krack_full, file='krack_full.dl', format="pajek")

# For a more general file type (e.g., importable to Excel),
# use the "edgelist" format. Note that neither of these will
# write the attributes; only the ties are maintained.
write.graph(krack_full, file='krack_full.txt', format="edgelist")
```

```
#########################################################################
# LAB 2: Methodological beginnings - Density, Reciprocity, Triads, #
# Transitivity, and heterogeneity. Node and network statistics.    #
#########################################################################


# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.
# Also see Lab 1 for prior functions.


#################################################################
#
# Lab 2
#
# The purpose of this lab is to acquire basic cohesion
# metrics of density, reciprocity, reach, path distance,
# and transitivity. In addition, we'll develop triadic
# analyses and a measure of ego-network heterogenity.
#
#################################################################



###
# 1. SET UP SESSION
###
install.packages("NetData")

library(igraph)
library(NetData)


###
# 2. LOAD DATA
###

# We would ordinarily need to follow the same proceedure we did for the Krackhardt data
# as we did in lab 1; see that lab for detail.

data(kracknets, package = "NetData")
```

```
# Reduce to non-zero edges and build a graph object
krack_full_nonzero_edges <- subset(krack_full_data_frame, (advice_tie > 0 | friendship_tie > 0 |
reports_to_tie > 0))
head(krack_full_nonzero_edges)

krack_full <- graph.data.frame(krack_full_nonzero_edges)
summary(krack_full)

# Set vertex attributes
for (i in V(krack_full)) {
	for (j in names(attributes)) {
		krack_full <- set.vertex.attribute(krack_full, j, index=i, attributes[i+1,j])
	}
}
summary(krack_full)

# Create sub-graphs based on edge attributes
krack_advice <- delete.edges(krack_full, E(krack_full)[get.edge.attribute(krack_full,name =
"advice_tie")==0])
summary(krack_advice)

krack_friendship <- delete.edges(krack_full, E(krack_full)[get.edge.attribute(krack_full,name =
"friendship_tie")==0])
summary(krack_friendship)

krack_reports_to <- delete.edges(krack_full, E(krack_full)[get.edge.attribute(krack_full,name =
"reports_to_tie")==0])
summary(krack_reports_to)



###
# 3. NODE-LEVEL STATISTICS
###

# Compute the indegree and outdegree for each node, first in the
# full graph (accounting for all tie types) and then in each
# tie-specific sub-graph.
deg_full_in <- degree(krack_full, mode="in")
deg_full_out <- degree(krack_full, mode="out")
deg_full_in
deg_full_out

deg_advice_in <- degree(krack_advice, mode="in")
```

```
deg_advice_out <- degree(krack_advice, mode="out")
deg_advice_in
deg_advice_out

deg_friendship_in <- degree(krack_friendship, mode="in")
deg_friendship_out <- degree(krack_friendship, mode="out")
deg_friendship_in
deg_friendship_out

deg_reports_to_in <- degree(krack_reports_to, mode="in")
deg_reports_to_out <- degree(krack_reports_to, mode="out")
deg_reports_to_in
deg_reports_to_out

# Reachability can only be computed on one vertex at a time. To
# get graph-wide statistics, change the value of "vertex"
# manually or write a for loop. (Remember that, unlike R objects,
# igraph objects are numbered from 0.)

reachability <- function(g, m) {
	reach_mat = matrix(nrow = vcount(g),
			ncol = vcount(g))
	for (i in 1:vcount(g)) {
		reach_mat[i,] = 0
		this_node_reach <- subcomponent(g, (i - 1), mode = m)

		for (j in 1:(length(this_node_reach))) {
			alter = this_node_reach[j] + 1
			reach_mat[i, alter] = 1
		}
	}
	return(reach_mat)
}

reach_full_in <- reachability(krack_full, 'in')
reach_full_out <- reachability(krack_full, 'out')
reach_full_in
reach_full_out

reach_advice_in <- reachability(krack_advice, 'in')
reach_advice_out <- reachability(krack_advice, 'out')
reach_advice_in
reach_advice_out
```

```
reach_friendship_in <- reachability(krack_friendship, 'in')
reach_friendship_out <- reachability(krack_friendship, 'out')
reach_friendship_in
reach_friendship_out

reach_reports_to_in <- reachability(krack_reports_to, 'in')
reach_reports_to_out <- reachability(krack_reports_to, 'out')
reach_reports_to_in
reach_reports_to_out


# Often we want to know path distances between individuals in a network.
# This is often done by calculating geodesics, or shortest paths between
# each ij pair. One can symmetrize the data to do this (see lab 1), or
# calculate it for outward and inward ties separately. Averaging geodesics
# for the entire network provides an average distance or sort of cohesiveness
# score. Dichotomizing distances reveals reach, and an average of reach for
# a network reveals what percent of a network is connected in some way.

# Compute shortest paths between each pair of nodes.
sp_full_in <- shortest.paths(krack_full, mode='in')
sp_full_out <- shortest.paths(krack_full, mode='out')
sp_full_in
sp_full_out

sp_advice_in <- shortest.paths(krack_advice, mode='in')
sp_advice_out <- shortest.paths(krack_advice, mode='out')
sp_advice_in
sp_advice_out

sp_friendship_in <- shortest.paths(krack_friendship, mode='in')
sp_friendship_out <- shortest.paths(krack_friendship, mode='out')
sp_friendship_in
sp_friendship_out

sp_reports_to_in <- shortest.paths(krack_reports_to, mode='in')
sp_reports_to_out <- shortest.paths(krack_reports_to, mode='out')
sp_reports_to_in
sp_reports_to_out


# Assemble node-level stats into single data frame for export as CSV.
```

```
# First, we have to compute average values by node for reachability and
# shortest path. (We don't have to do this for degree because it is
# already expressed as a node-level value.)
reach_full_in_vec <- vector()
reach_full_out_vec <- vector()
reach_advice_in_vec <- vector()
reach_advice_out_vec <- vector()
reach_friendship_in_vec <- vector()
reach_friendship_out_vec <- vector()
reach_reports_to_in_vec <- vector()
reach_reports_to_out_vec <- vector()

sp_full_in_vec <- vector()
sp_full_out_vec <- vector()
sp_advice_in_vec <- vector()
sp_advice_out_vec <- vector()
sp_friendship_in_vec <- vector()
sp_friendship_out_vec <- vector()
sp_reports_to_in_vec <- vector()
sp_reports_to_out_vec <- vector()

for (i in 1:vcount(krack_full)) {
	reach_full_in_vec[i] <- mean(reach_full_in[i,])
	reach_full_out_vec[i] <- mean(reach_full_out[i,])
	reach_advice_in_vec[i] <- mean(reach_advice_in[i,])
	reach_advice_out_vec[i] <- mean(reach_advice_out[i,])
	reach_friendship_in_vec[i] <- mean(reach_friendship_in[i,])
	reach_friendship_out_vec[i] <- mean(reach_friendship_out[i,])
	reach_reports_to_in_vec[i] <- mean(reach_reports_to_in[i,])
	reach_reports_to_out_vec[i] <- mean(reach_reports_to_out[i,])

	sp_full_in_vec[i] <- mean(sp_full_in[i,])
	sp_full_out_vec[i] <- mean(sp_full_out[i,])
	sp_advice_in_vec[i] <- mean(sp_advice_in[i,])
	sp_advice_out_vec[i] <- mean(sp_advice_out[i,])
	sp_friendship_in_vec[i] <- mean(sp_friendship_in[i,])
	sp_friendship_out_vec[i] <- mean(sp_friendship_out[i,])
	sp_reports_to_in_vec[i] <- mean(sp_reports_to_in[i,])
	sp_reports_to_out_vec[i] <- mean(sp_reports_to_out[i,])
}

# Next, we assemble all of the vectors of node-levelvalues into a
```

```
# single data frame, which we can export as a CSV to our working
# directory.
node_stats_df <- cbind(deg_full_in,
                       deg_full_out,
                       deg_advice_in,
                       deg_advice_out,
                       deg_friendship_in,
                       deg_friendship_out,
                       deg_reports_to_in,
                       deg_reports_to_out,

                       reach_full_in_vec,
                       reach_full_out_vec,
                       reach_advice_in_vec,
                       reach_advice_out_vec,
                       reach_friendship_in_vec,
                       reach_friendship_out_vec,
                       reach_reports_to_in_vec,
                       reach_reports_to_out_vec,

                       sp_full_in_vec,
                       sp_full_out_vec,
                       sp_advice_in_vec,
                       sp_advice_out_vec,
                       sp_friendship_in_vec,
                       sp_friendship_out_vec,
                       sp_reports_to_in_vec,
                       sp_reports_to_out_vec)

write.csv(node_stats_df, 'krack_node_stats.csv')

# Question #1 - What do these statistics tell us about
# each network and its individuals in general?

###
# 3. NETWORK-LEVEL STATISTICS
###

# Many initial analyses of networks begin with distances and reach,
# and then move towards global summary statistics of the network.
#
# As a reminder, entering a question mark followed by a function
# name (e.g., ?graph.density) pulls up the help file for that function.
```

```
# This can be helpful to understand how, exactly, stats are calculated.

# Degree
mean(deg_full_in)
sd(deg_full_in)
mean(deg_full_out)
sd(deg_full_out)

mean(deg_advice_in)
sd(deg_advice_in)
mean(deg_advice_out)
sd(deg_advice_out)

mean(deg_friendship_in)
sd(deg_friendship_in)
mean(deg_friendship_out)
sd(deg_friendship_out)

mean(deg_reports_to_in)
sd(deg_reports_to_in)
mean(deg_reports_to_out)
sd(deg_reports_to_out)



# Shortest paths
# ***Why do in and out come up with the same results?
# In and out shortest paths are simply transposes of one another;
# thus, when we compute statistics across the whole network they have to be the same.

mean(sp_full_in[which(sp_full_in != Inf)])
sd(sp_full_in[which(sp_full_in != Inf)])
mean(sp_full_out[which(sp_full_out != Inf)])
sd(sp_full_out[which(sp_full_out != Inf)])

mean(sp_advice_in[which(sp_advice_in != Inf)])
sd(sp_advice_in[which(sp_advice_in != Inf)])
mean(sp_advice_out[which(sp_advice_out != Inf)])
sd(sp_advice_out[which(sp_advice_out != Inf)])

mean(sp_friendship_in[which(sp_friendship_in != Inf)])
sd(sp_friendship_in[which(sp_friendship_in != Inf)])
mean(sp_friendship_out[which(sp_friendship_out != Inf)])
sd(sp_friendship_out[which(sp_friendship_out != Inf)])
```

```
mean(sp_reports_to_in[which(sp_reports_to_in != Inf)])
sd(sp_reports_to_in[which(sp_reports_to_in != Inf)])
mean(sp_reports_to_out[which(sp_reports_to_out != Inf)])
sd(sp_reports_to_out[which(sp_reports_to_out != Inf)])

# Reachability
mean(reach_full_in[which(reach_full_in != Inf)])
sd(reach_full_in[which(reach_full_in != Inf)])
mean(reach_full_out[which(reach_full_out != Inf)])
sd(reach_full_out[which(reach_full_out != Inf)])

mean(reach_advice_in[which(reach_advice_in != Inf)])
sd(reach_advice_in[which(reach_advice_in != Inf)])
mean(reach_advice_out[which(reach_advice_out != Inf)])
sd(reach_advice_out[which(reach_advice_out != Inf)])

mean(reach_friendship_in[which(reach_friendship_in != Inf)])
sd(reach_friendship_in[which(reach_friendship_in != Inf)])
mean(reach_friendship_out[which(reach_friendship_out != Inf)])
sd(reach_friendship_out[which(reach_friendship_out != Inf)])

mean(reach_reports_to_in[which(reach_reports_to_in != Inf)])
sd(reach_reports_to_in[which(reach_reports_to_in != Inf)])
mean(reach_reports_to_out[which(reach_reports_to_out != Inf)])
sd(reach_reports_to_out[which(reach_reports_to_out != Inf)])

# Density
graph.density(krack_full)
graph.density(krack_advice)
graph.density(krack_friendship)
graph.density(krack_reports_to)

# Reciprocity
reciprocity(krack_full)
reciprocity(krack_advice)
reciprocity(krack_friendship)
reciprocity(krack_reports_to)

# Transitivity
transitivity(krack_full)
transitivity(krack_advice)
transitivity(krack_friendship)
```

```
transitivity(krack_reports_to)

# Triad census. Here we'll first build a vector of labels for
# the different triad types. Then we'll combine this vector
# with the triad censuses for the different networks, which
# we'll export as a CSV.

census_labels = c('003',
                  '012',
                  '102',
                  '021D',
                  '021U',
                  '021C',
                  '111D',
                  '111U',
                  '030T',
                  '030C',
                  '201',
                  '120D',
                  '120U',
                  '120C',
                  '210',
                  '300')
tc_full <- triad.census(krack_full)
tc_advice <- triad.census(krack_advice)
tc_friendship <- triad.census(krack_friendship)
tc_reports_to <- triad.census(krack_reports_to)

triad_df <- data.frame(census_labels,
                       tc_full,
                       tc_advice,
                       tc_friendship,
                       tc_reports_to)
triad_df

# To export any of these vectors to a CSV for use in another program, simply
# use the write.csv() command:
write.csv(triad_df, 'krack_triads.csv')

# Question #2 - (a) How do the three networks differ on network statictics?
# (b) What does the triad census tell us? Can you calculate the likelihood of
# any triad's occurrence? (c) See the back of Wasserman and Faust and its section
# on triads. Calculate the degree of clustering and hierarchy in Excel.
```

```
# What do we learn from that?




###
# 4. HETEROGENEITY
###

# Miller and McPherson write about processes of homophily and
# here we take a brief look at one version of this issue.
# In particular, we look at the extent to which each actor's
# "associates" (friend, advisor, boos) are heterogenous or not.

# We'll use a statistic called the IQV, or Index of Qualitative
# Variation. This is just an implementation of Blau's Index of
# Heterogeneity (known to economists as the Herfindahl-Hirschman
# index), normalized so that perfect heterogeneity (i.e., equal
# distribution across categories) equals 1.

# NOTE that this code only works with categorical variables that
# have been numerically coded to integer values that ascend
# sequentially from 0; you may have to recode your data to get this
# to work properly.
# We are interested in many of the attributes of nodes.  To save
# time and to make our lives better we are going to create a function
# that will provide an IQV statistic for any network and for
# any categorical variable.  A function is a simple way to
# create code that is both reusable and easier to edit.

# Functions have names and receive arguments.  For example,
# anytime you call table() you are calling the table function.
# We could write code to duplicate the table function for each
# of our variables, but it is faster to write a single general tool
# that will provide frequencies for any variable. If I have
# a dataframe with the variable gender and I want to see the
# split of males and females I would pass the argument
# "dataframe$gender" to the table function. We follow a
# similar model here. Understanding each step is less important
# than understanding the usefulness and power of functions.

get_iqvs <- function(graph, attribute) {
```

```
#we have now defined a function, get_iqvs, that will take the
# graph "graph" and find the iqv statistic for the categorical
# variable "attribute." Within this function whenever we use the
#variables graph or attribute they correspond to the graph and
# variable we passed (provided) to the function

mat <- get.adjacency(graph)

# To make this function work on a wide variety of variables we
# find out how many coded levels (unique responses) exist for
# the attribute variable programatically

    attr_levels = get.vertex.attribute(graph,
                                attribute,
                                V(graph))

    num_levels = length(unique(attr_levels))
    iqvs = rep(0, nrow(mat))

# Now that we know how many levels exist we want to loop
# (go through) each actor in the network. Loops iterate through
# each value in a range.  Here we are looking through each ego
# in the range of egos starting at the first and ending at the
# last.  The function nrow provides the number of rows in an
# object and the ":" opperand specifies the range.  Between
# the curly braces of the for loop ego will represent exactly
# one value between 1 and the number of rows in the graph
# object, iterating by one during each execution of the loop.

    for (ego in 1:nrow(mat)) {

        # initialize actor-specific variables
        alter_attr_counts = rep(0, num_levels)
        num_alters_this_ego = 0
        sq_fraction_sum = 0

# For each ego we want to check each tied alter for the same
# level on the variable attribute as the ego.

        for (alter in 1:ncol(mat)) {

            # only examine alters that are actually tied to ego
            if (mat[ego, alter] == 1) {
```

```
                num_alters_this_ego = num_alters_this_ego + 1

                # get the alter's level on the attribute
                alter_attr = get.vertex.attribute(graph,
                   attribute, (alter - 1))

                # increment the count of alters with this level
                # of the attribute by 1
                alter_attr_counts[alter_attr + 1] =
                   alter_attr_counts[alter_attr + 1] + 1
            }
        }

        # now that we're done looping through all of the alters,
        # get the squared fraction for each level of the attribute
        # out of the total number of attributes
        for (i in 1:num_levels) {
            attr_fraction = alter_attr_counts[i] /
               num_alters_this_ego
            sq_fraction_sum = sq_fraction_sum + attr_fraction ^ 2
        }

        # now we can compute the ego's blau index...
        blau_index = 1 - sq_fraction_sum

        # and the ego's IQV, which is just a normalized blau index
        iqvs[ego] = blau_index / (1 - (1 / num_levels))
    }

# The final part of a function returns the calculated value.
#  So if we called get_iqvs(testgraph, gender) return would
# provide the iqvs for gender in the test graph.  If we are also
# intersted in race we could simply change the function call
# to get_iqvs(testgraph, race).  No need to write all this
# code again for different variables.

    return(iqvs)
}



# For this data set, we'll look at homophily across departments,
```

```
# which is already coded 0-4, so no recoding is needed.

advice_iqvs <- get_iqvs(krack_advice, 'DEPT')
advice_iqvs

friendship_iqvs <- get_iqvs(krack_friendship, 'DEPT')
friendship_iqvs

reports_to_iqvs <- get_iqvs(krack_reports_to, 'DEPT')
reports_to_iqvs

# Question #3 - What does the herfindahl index reveal about
# attribute sorting in networks? What does it mean for each network?



#####
# Extra-credit: What might be a better way to test the occurrence
# of homophily or segregation in a network? How might we code that in R?
#####

#####
# Tau statistic (code by Sam Pimentel)
#####



#R code for generating random graphs:
#requires packages ergm, intergraph

#set up weighting vectors for clustering and hierarchy
clust.mask <- rep(0,16)
clust.mask[c(1,3,16)] <- 1
hier.mask <- rep(1,16)
hier.mask[c(6:8,10:11)]  <- 0

#compute triad count and triad proportion for a given weighting vector
mask.stat <- function(my.graph, my.mask){
   n.nodes <- vcount(my.graph)
   n.edges <- ecount(my.graph)
   #set probability of edge formation in random graph to proportion of possible edges present in original
   p.edge <- n.edges/(n.nodes*(n.nodes +1)/2)
   r.graph <- as.network.numeric(n.nodes, density = p.edge)
   r.igraph <- as.igraph(r.graph)
   tc.graph <- triad.census(r.igraph)
```

```
  clust <- sum(tc.graph*my.mask)
  clust.norm <- clust/sum(tc.graph)
  return(c(clust,clust.norm))
}

#build 100 random graphs and compute their clustering and hierarchy measurements to create an
empirical null distribution
emp.distro <- function(this.graph){
 clust <- matrix(rep(0,200), nrow=2)
 hier <- matrix(rep(0,200),nrow=2)
 for(i in c(1:100)){
   clust[,i] <- mask.stat(this.graph, clust.mask)
   hier[,i] <- mask.stat(this.graph, hier.mask)
 }
 my.mat <- rbind(clust, hier)
 rownames(my.mat) <- c("clust.ct", "clust.norm", "hier.ct", "hier.ct.norm")
 return(my.mat)
}

#fix randomization if desired so results are replicable
#set.seed(3123)
#compute empirical distributions for each network
hc_advice <- emp.distro(krack_advice)
hc_friend <- emp.distro(krack_friendship)
hc_report <- emp.distro(krack_reports_to)

#find empirical p-value
get.p <- function(val, distro)
{
      distro.n <- sort(distro)
      distro.n <- distro.n - median(distro.n)
      val.n <- val - median(distro.n)
      p.val <- sum(abs(distro.n) > abs(val.n))/100
      return(p.val)
}
get.p(198, hc_full[1,])
get.p(194, hc_advice[1,])
get.p(525, hc_friend[1,])
get.p(1003, hc_report[1,])
get.p(979, hc_full[3,])
get.p(1047, hc_advice[3,])
get.p(1135, hc_friend[3,])
get.p(1314, hc_report[3,])
```

#generate 95% empirical confidence intervals for triad counts

#clustering
c(sort(hc_advice[1,])[5], sort(hc_advice[1,])[95])
c(sort(hc_friend[1,])[5], sort(hc_friend[1,])[95])
c(sort(hc_report[1,])[5], sort(hc_report[1,])[95])

#hierarchy
c(sort(hc_advice[3,])[5], sort(hc_advice[3,])[95])
c(sort(hc_friend[3,])[5], sort(hc_friend[3,])[95])
c(sort(hc_report[3,])[5], sort(hc_report[3,])[95])

Skip to Content

STANFORD UNIVERSITY | SOCIAL NETWORK ANALYSIS

- Home
- R labs
- Courses
- Members
- Reseach & Resources

Organizations and Networks Courses

Home » Courses

Social Network Analysis (McFarland – Soc 369m / Ed 316x)
http://ed.stanford.edu/suse/dan-mcfarland/NetAnlSyl.htm

Workshop: Networks and Organizations (McFarland / Powell – Ed 361 / Soc 361W)
http://**sna.stanford/edu**/schedule.html

Network Analysis (Leskovec - CS 322)
Social Networks, Careers and Markets (Sorenson / Soule - GSBGEN 534)

Social Networks - Theory, Methods, and Applications (Lifschitz - MS&E 189)

Network Structures and Analysis (MS&E 238)

Social and Economic Networks (Jackson - Econ 291)
http://www.stanford.edu/~jacksonm/291syllabus.pdf

Introduction to Social Networks (Parigi Soc 126/226)
http://www.stanford.edu/~hhillman/Syllabus_Soc126_Fall_07_08.pdf

Introduction to Social Network Analysis (Parigi Soc 128/228)
http://www.stanford.edu/~hhillman/Soc%20324_winter08.pdf

- Stanford University

```
########################################
# LAB 3: Clusters, Factions and Cores #
########################################

# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.


######################################################################
#
# Lab 3
#
# The purpose of this lab is to identify friendship groups
# or cliques using different methods, to discern the best
# fitting clique structure, and to develop clique-models of
# task and social interaction patterns.
#
######################################################################


###
#1. SETUP
###

# For this lab, we'll use a few different packages. "sna" is a
# social-network-analysis package similar to igraph, but with some
# different and useful functions. "cluster" is a generic cluster-
# analysis package with applications beyond the social-network
# context. "animation" is used, not surprisingly, to produce
# animations.

install.packages("animation", repos = "http://cran.cnr.berkeley.edu/", dependencies = TRUE)
library('igraph')
library('cluster')
library('animation')


###
#2. LOADING AND FORMATTING DATA
###

# We'll use the NetData package to load the data this time around:
```

```
data(studentnets.M182, package = "NetData")

# For this lab, we'll use three files from Student Nets M182 Sem
# 2. FRN.DAT shows self-reported friendship ties, SSL.DAT shows
# observed social interactions, and TSL.DAT shows observed
# task interactions.
#
# All of the files should be in the following format:
#       1       1 0.000000
#       1       2 2.000000
#       1       3 1.000000
# where the first column is the ego, the second column is the alter,
# and the third column is an integer or floating-point number
# greater than or equal to zero showing the strength of the
# association for the given two vertices.

# Reduce to non-zero edges and build a graph object
m182_full_nonzero_edges <- subset(m182_full_data_frame, (friend_tie > 0 | social_tie > 0 | task_tie >
0))
head(m182_full_nonzero_edges)

m182_full <- graph.data.frame(m182_full_nonzero_edges)
summary(m182_full)

# Create sub-graphs based on edge attributes
m182_friend <- delete.edges(m182_full, E(m182_full)[get.edge.attribute(m182_full,name = "friend_tie")
==0])

m182_social <- delete.edges(m182_full, E(m182_full)[get.edge.attribute(m182_full,name = "social_tie")
==0])
summary(m182_social)

m182_task <- delete.edges(m182_full, E(m182_full)[get.edge.attribute(m182_full,name = "task_tie")
==0])
summary(m182_task)

# Look at the plots for each sub-graph
friend_layout <- layout.fruchterman.reingold(m182_friend)
plot(m182_friend, layout=friend_layout, edge.arrow.size=.5)

social_layout <- layout.fruchterman.reingold(m182_social)
plot(m182_social, layout=social_layout, edge.arrow.size=.5)
```

```
task_layout <- layout.fruchterman.reingold(m182_task)
plot(m182_task, layout=task_layout, edge.arrow.size=.5)


###
# 3. COMMUNITY DETECTION
###

# We'll use the friend sub-graph as the basis for our community
# detection methods. For clarity and simplicity, we'll set the
# network to undirected and remove isolated vertices. Comparing
# m182_friend before and after these operations, you'll notice
# that the number of edges decreases as reciprocated directed ties
# are consolidated into single undirected ties, and the number of
# vertices decreases as isolates are removed.
m182_friend_und <- as.undirected(m182_friend, mode='collapse')
m182_friend_no_iso <- delete.vertices(m182_friend_und, V(m182_friend_und)[degree
(m182_friend_und)==0])
summary(m182_friend)
summary(m182_friend_no_iso)

# There are many different ways to detect communities. In this
# lab, we'll use three: hierarchical NetCluster, walktrap, and
# edge-betweenness. As you use them, consider how they portray
# clusters and consider which one(s) afford a sensible view of
# the social world as cohesively organized.


###
# 3A. COMMUNITY DETECTION: WALKTRAP
###

# This algorithm detects communities through a series of short
# random walks, with the idea that the vertices encountered on
# any given random walk are more likely to be within a community
# than not. The algorithm initially treats all nodes as
# communities of their own, then merges them into larger
# communities, and these into still larger communities, and so on.
# In each step a new community is created from two other
# communities, and its ID will be one larger than the largest
# community ID so far. This means that before the first merge we
# have n communities (the number of vertices in the graph)
```

```
# numbered from zero to n-1. The first merge creates community n,
# the second community n+1, etc. This merge history is returned by
# the function.

# The Walktrap algorithm calls upon the user to specify the length of random walks,
# and Pons and Latapy (2005) recommend walks of 4 or 5 steps. However, Waugh
# et al found that for many groups (Congresses), these lengths did not provide the
# maximum modularity score (this is what I assume to be the numbers on the left
# hand side of the dendrogram—I can't figure it out. If it's not this, we should #
# output the modularity score and plot that [same for edge betweenness – what
# Newman-Girvan do]). To be thorough in their attempts to optimize modularity,
# they performed the walktrap algorithm 50 times for each group (using random
# walks of lengths 1–50) and selected the network partition with the highest
# modularity value from those 50. They call this the "maximum modularity
# partition" and insert the parenthetical "(though, strictly speaking, this cannot be
# proven to be the optimum without computationally-prohibitive exhaustive
# enumeration (Brandes et al. 2008))."


friend_comm_wt <- walktrap.community(m182_friend_no_iso, steps=200,modularity=TRUE,
labels=TRUE)
friend_comm_wt

# As with hierarchical NetCluster above, we can also visualize
# the clusters generated by walktrap as a dendrogram (but note that
# the clusters themselves may be different). Here, the y-axis
# reflects the distance metric used by the walktrap algorithm; for
# more on this, see Pascal Pons, Matthieu Latapy: Computing communities
# in large networks using random walks, http://arxiv.org/abs/physics/0512106.
friend_comm_dend <- as.dendrogram(friend_comm_wt, use.modularity=TRUE)
plot(friend_comm_dend)

# Question #2 - How many clusters would you select here and why?


###
# 3B. COMMUNITY DETECTION: EDGE BETWEENNESS METHOD
###

# The edge-betweenness score of an edge measures the number of
# shortest paths from one vertex to another that go through it.
# The idea of the edge-betweenness based community structure
# detection is that it is likely that edges connecting separate
```

```
# cluster have high edge-betweenness, as all the shortest paths
# from one cluster to another must traverse through them. So if we
# iteratively remove the edge with the highest edge-betweenness
# score we will get a hierarchical map of the communities in the
# graph.

# The idea of the edge betweenness based community structure detection is that it
# is likely that edges connecting separate modules have high edge betweenness as # all the shortest paths from one module to another must traverse through them.
# So if we gradually remove the edge with the highest edge betweenness score we
# will get a hierarchical map, a rooted tree, called a dendrogram of the graph. The
# leafs of the tree are the individual vertices and the root of the tree represents the
# whole graph.
# The Dendrogram we made seems to show values on the left for the average
# number of inter-community edges per vertex. Since the algorithm takes out the
# most between edges first, it forms cliques and leaves fewer and fewer inter-
# community edges in place.




# The following function will find the betweenness for each
# vertex.
friend_comm_eb <- edge.betweenness.community(m182_friend_no_iso)
friend_comm_eb

# This process also lends itself to visualization as a dendrogram.
# The y-axis reflects the distance metric used by the edge betweennes
# algorithm; for more on this, see M Newman and M Girvan: Finding and
# evaluating community structure in networks, Physical Review E 69, 026113
# (2004), http://arxiv.org/abs/cond-mat/0308217.
plot(as.dendrogram(friend_comm_eb))

# Question #3 - How many clusters would you select here and why?



# The following code produces an animation of the edge-betweeness
# process. It's easiest to run all of this code at once. The
# result is a set of .png files that will be saved to the default
# working directory (or the working directory specified by setwd(),
# if any). Note that the code may throw an error, but this doesn't
# necessarily mean it didn't work; check the appropriate folder
# for the .png files to see. Highlight all the .png files and open
```

```
# them in Preview as a slideshow.

# Before running this code, you need to install ImageMagick:
# http://www.imagemagick.org/script/binary-releases.php
# Scroll down to the Windows/Mac section (you probably do not want
# the Unix files at the top.)

# *** START ANIMATION CODE ***

jitter.ani <-function(x, g){

  l <- layout.kamada.kawai(g, niter=1000)
  ebc <- edge.betweenness.community(g)

  colbar <- rainbow(6)
  colbar2 <- c(rainbow(5), rep("black",15))

  for (i in 1:x) {
    g2 <- delete.edges(g, ebc$removed.edges[seq(length=i-1)])
    eb <- edge.betweenness(g2)
    cl <- clusters(g2)$membership
    q <- modularity(g, cl)
    E(g2)$color <- "grey"
    E(g2)[ order(eb, decreasing=TRUE)[1:5]-1 ]$color <- colbar2[1:5]

    E(g2)$width <- 1
    E(g2)[ color != "grey" ]$width <- 2

    plot(g2, layout=l, vertex.size=12,
      edge.label.color="red", vertex.color=colbar[cl+2],
      edge.label.font=2)
    title(main=paste("Q=", round(q,3)), font=2)
    ty <- seq(1,by=-strheight("1")*1.5, length=20)
    text(-1.3, ty, adj=c(0,0.5), round(sort(eb, dec=TRUE)[1:20],2),
      col=colbar2, font=2)
  }
}

saveMovie(jitter.ani(20, m182_friend_no_iso), interval = 0.5, outdir = getwd())

# *** END ANIMATION CODE ***
```

```
# Now we'll decide on an optimum number of communities for this
# network. All of the community methods above allow us to iterate
# through different numbers of communities and judge how well
# correlated a given idealized community structure is with the
# observed network.
#
# We'll start by getting an adjacency matrix based on the network
# with isolates removed.
friend_adj_mat_no_iso <- get.adjacency(m182_friend_no_iso, binary=TRUE)
friend_adj_mat_no_iso
num_vertices = nrow(friend_adj_mat_no_iso)
num_vertices

# Next, we'll derive a set of idealized community structures
# at each possible number of communities. We can correlate each
# of these structures with the observed community and store
# the correlation statistics in a vector.
#
# This code loops through each possible number of communities that
# can be derived using the edge-betweenness method, but it could
# readily be applied to the other clustering methods above. For each
# number of communities, it prints out an adjacency matrix with 0s
# indicating that the row and column vertices do not share a
# community and 1s indicating that they do.
#
# Thus, the first iteration returns 14 communities, one for each
# vertex, and an adjacency matrix of all 0s. The last iteration
# returns a single community comprised of all vertices, and an
# adjacency matrix of all 1s (except on the diagonal).
#
# Meanwhile, we can correlate each community structure with the
# observed community to generate a list of ideal-observed
# correlations.
library(sna)
ideal_observed_cors = vector()
for (i in 0:(num_vertices-1)) {
	num_comms = (num_vertices - i)
	cat('number of communities: ', num_comms, '\n')
	community <- community.to.membership(m182_friend_no_iso, friend_comm_eb$merges, i)
	print(community)
	idealized_comm_mat <- matrix(nrow=num_vertices, ncol=num_vertices)

	for (m in 1:num_vertices) {
```

```
            for (n in 1:num_vertices) {
                if (m==n) {
                    idealized_comm_mat[m,n] = 0
                } else if (community$membership[m] == community$membership[n]) {
                    idealized_comm_mat[m,n] = 1
                } else {
                    idealized_comm_mat[m,n] = 0
                }
            }
        }
        print(idealized_comm_mat)

        if (num_comms > 1 & num_comms < num_vertices) {
            ideal_observed_cors <- append(ideal_observed_cors, (gcor(idealized_comm_mat,
friend_adj_mat_no_iso)))
            print(ideal_observed_cors[length(ideal_observed_cors)])
        } else {
            ideal_observed_cors <- append(ideal_observed_cors, 0)
            print('unable to calcuate correlation; setting value to 0')
        }
        cat('\n')
}
ideal_observed_cors <- rev(ideal_observed_cors)
ideal_observed_cors

# We can then plot the list of correlations to determine the
# optimum number of communities for this particular network.
plot(ideal_observed_cors)

# QUESTION #4 - Which clustering technique affords the best clique
# solution and why? What supports your claim? Why use say 4 clusters
# instead of 10? What evidence supports the claim for # of clusters?


###
# 4. BLOCKMODELING
###


# Now we'll look at some community statistics across different
# relationship networks. For all of these, we'll use the three
# edge-betweenness communities generated above as our basic
# social structure. We will reintroduce the two isolates we
```

```
# removed, assigning each to his/her own "community."
#
# First, we'll set a vector of communities, ordered according
# to the order of vertices.
communities <- community.to.membership(m182_friend_no_iso, friend_comm_eb$merges,
num_vertices-3)
communities

# Now we have to manually insert communities corresponding to the
# isolates. We can get their vertex ID numbers via:
friend_adj_mat_full <- get.adjacency(m182_friend, binary=TRUE)
which(degree(friend_adj_mat_full)==0)

# Insert the isolate communities at indices 4 and 16.
# IN ORDER TO PERFORM THE COMMUNITY DETECTION ALGORITHMS, WE HAD TO
REMOVE THE ISOLATES FROM THE GRAPH (HENCE THE NO_ISO MATRICES ABOVE).
NOW WE WANT TO RE-INSERT THEM, SO WE ASSIGNING EACH TO ITS OWN
"COMMUNITY" (NUMBERED 3 AND 4) AND MANUALLY ADD THESE COMMUNITIES TO
THE VECTOR OF COMMUNITIES IN THE APPROPRIATE PLACE.
communities_full <- vector()
communities_full <- append(communities_full, communities$membership[1:3])
communities_full <- append(communities_full, 3)
communities_full <- append(communities_full, communities$membership[4:14])
communities_full <- append(communities_full, 4)
communities_full

# We can use this community membership vector to produce a
# blockmodel, showing the within- and between-cluster densities.
# For this we'll use the blockmodel() function in sna. Unlike
# igraph, sna counts from 1, so the first thing we need to do
# is renumber the IDs in the communities vector.
communities_full <- communities_full+1
communities_full

# Now we can generate the blockmodel using the adjacency matrix
# and community vector. Each "block" corresponds to a community
# listed in the communities_full vector; the values are the
# densities within and between communities.

friend_blockmodel <- blockmodel(friend_adj_mat_full, communities_full)
friend_blockmodel

# Compare the density values from the blockmodel to the overall
```

```
# density of the network.
graph.density(m182_friend)

# Now we can repeat the blockmodel process to get within- and
# between-cluster densities for the social-interaction network,
# using the communities generated with the friends network for our
# clusters.
#
# For this data set, we want to retain the tie values.
social_adj_mat <- get.adjacency(m182_social, attr='social_tie')
social_adj_mat
social_blockmodel <- blockmodel(social_adj_mat, communities_full)
social_blockmodel

# Because we retatined tie values above, we should compare the
# blockmodel values not to the graph's overall density but to its
# overall mean.
social_mean <- mean(social_adj_mat)
social_mean

# We can repeat one more time for the task-interaction network,
# once again retaining valued ties.
task_adj_mat <- get.adjacency(m182_task, attr='task_tie')
task_adj_mat
task_blockmodel <- blockmodel(task_adj_mat, communities_full)
task_blockmodel

task_mean <- mean(task_adj_mat)
task_mean

# QUESTION #5 - Using the average density for each type of tie as
# an alpha-cutoff level, answer the following: (a) How do friendship
# cliques shape patterns of task and social interaction? (b) Are task
# or social interactions following clique boundaries? Why or why not
# might this happen?



###
# 5. K-CORES
###

# The graph.coreness() function in igraph returns a vector containing
# the degree of the highest-degree k-core to which each vertex
```

```
# belongs.
coreness = graph.coreness(m182_task)
coreness

# Note that the output of graph.coreness refers simply to the *degree*
# of the k-core, not to the k-core itself; thus, two vertices both with
# coreness of 3 may not be connected at all and thus may be in separate
# k-cores.
#
# One way to get a sense of the actual k-core structure is to simply
# plot the graph and color-code by k-core:
make_k_core_plot <- function (g) {
      lay1 <- layout.fruchterman.reingold(g)
      plot(g,
          vertex.color = graph.coreness(g),
          layout=lay1,
          edge.arrow.size = .5)
}

make_k_core_plot(m182_friend)
make_k_core_plot(m182_social)
make_k_core_plot(m182_task)

# Here's an artificial example showing two separate 3-cores:
g1 <- graph.ring(10)
g1 <- add.edges(g1, c(0,2, 1,3, 0,3, 5,7, 5,8, 6,8))
make_k_core_plot(g1)

# Question #6 - What's the difference between K-cores and cliques?
# Why might one want to use one over the other?



###
# EXTRA CREDIT:
# igraph's built-in functionality merely shows the the degree of the
# highest-degree k-core to which each vertext belongs. However,
# vertices with the same coreness may be part of different k-cores,
# as shown in the artifical 3-core example above.
#
# Can you figure out a way to to generate not only the coreness of
# each vertex, but also an indicator of which specific k-core the
# vertex belongs to (and thus which other vertices are in the same
# k-core)?
```

###

```
######################
# LAB 4: Centrality #
######################


# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.


#################################################################
#
# Lab 4
#
# The purpose of this lab is to acquire centrality measures,
# to determine how they are interrelated, and to discern
# what they mean.
#
#################################################################


###
# 1. SETUP
###
library(igraph)


###
# 2. LOAD DATA
###

# This lab uses SSL.dat (social interaction) and TSL.dat (task
# interaction) from the S641 Semester 1 class in student_nets.
# The class is a biology 2 class at a public high school.

# load data:
data(studentnets.S641, package = "NetData")

# Reduce to non-zero edges and build a graph object
s641_full_nonzero_edges <- subset(s641_full_data_frame, (social_tie > 0 | task_tie > 0))
head(s641_full_nonzero_edges)

s641_full <- graph.data.frame(s641_full_nonzero_edges)
```

```
summary(s641_full)

# Create sub-graphs based on edge attributes and remove isolates
s641_social <- delete.edges(s641_full, E(s641_full)[get.edge.attribute(s641_full,name = "social_tie")
==0])
s641_social <- delete.vertices(s641_social, V(s641_social)[degree(s641_social)==0])
summary(s641_social)

s641_task <- delete.edges(s641_full, E(s641_full)[get.edge.attribute(s641_full,name = "task_tie")==0])
s641_task <- delete.vertices(s641_task, V(s641_task)[degree(s641_task)==0])
summary(s641_task)

# Look at the plots for each sub-graph
social_layout <- layout.fruchterman.reingold(s641_social)
plot(s641_social, layout=social_layout, edge.arrow.size=.5)

# Note: click on the graph and then use the drop down menu to
# save any plot you like -- it will save as a pdf.

task_layout <- layout.fruchterman.reingold(s641_task)
plot(s641_task, layout=task_layout, edge.arrow.size=.5)

# Question #1 - what can you say about network centralization from these graphs?



###
# 3. CALCULATE CENTRALITY MEASURES FOR SOCIAL
###

# Indegree centrality measures how many people direct social
# talk to the individual.
indegree_social <- degree(s641_social, mode='in')
indegree_social

# Outdegree centrality measures how many people the actor directs
# social talk to.
outdegree_social <- degree(s641_social, mode='out')
outdegree_social

# Closeness is the mean geodesic distance between a given node and
# all other nodes with paths from the given node to the other
# node. This is close to being the mean shortest path, but
# geodesic distances give higher values for more central nodes.
```

```
#
# In a directed network, we can think of in-closeness centrality
# as the average number of steps one would have to go through to
# get TO a given node FROM all other reachable nodes in the
# network. Out-closeness centrality, not surprisingly, measures
# the same thing with the directionality reversed.

# In-closeness centrality
incloseness_social <- closeness(s641_social, mode='in')
incloseness_social

# Out-closeness
outcloseness_social <- closeness(s641_social, mode='out')
outcloseness_social

# Betweenness centrality measures the number of shortest paths
# going through a specific vertex; it is returned by the
# betweenness() function. (Recall that in the previous lab we used
# a related measure called edge betweenness, which is returned by
# the edge.betweenness() function.)
betweenness_social <- betweenness(s641_social)
betweenness_social

# Eigenvector centrality gives greater weight to a node the more
# it is connected to other highly connected nodes. A node
# connected to five high-scoring nodes will have higher
# eigenvector centrality than a node connected to five low-scoring
# nodes. Thus, it is often interpreted as measuring a node's
# network importance.
#
# In directed networks, there are 'In' and 'Out' versions. In
# information flow studies, for instance, In-Eigenvector scores
# would reflect which nodes are high on receiving information,
# while Out-Eigenvector scores would reflect which nodes are high
# on broadcasting information.
#
# For these data, we will simply symmetrize to generate an
# undirected eigenvector centrality score.
#
# Note that, unlike the other centrality measures, evcent()
# returns a complex object rather than a simple vector. Thus,
# we need to first get the evcent() output and then select the
# eigenvector scores from it.
```

```
s641_social_undirected <- as.undirected(s641_social, mode='collapse')
ev_obj_social <- evcent(s641_social_undirected)
eigen_social <- ev_obj_social$vector
eigen_social

#####
# Extra Credit - what code would you write in R
# to get the directed versions of eigenvector centrality?
#####

# To get the summary table, we'll construct a data frame with
# the vertices as rows and the centrality scores as columns.
#
# Note that the vertex IDs are NOT the same as the first column
# of row numbers. This is because we previously removed isolates.
central_social <- data.frame(V(s641_social)$name, indegree_social, outdegree_social,
incloseness_social, outcloseness_social, betweenness_social, eigen_social)
central_social

# Now we'll examine the table to find the most central actors
# according to the different measures we have. When looking at
# each of these measures, it's a good idea to have your plot on
# hand so you can sanity-check the results.
plot(s641_social, vertex.size=10, vertex.label=V(s641_social)$name,
edge.arrow.size = 0.5, layout=layout.fruchterman.reingold,main='Classroom S641 Social Talk')

# Show table sorted by decreasing indegree. The order() function
# returns a vector in ascending order; the minus sign flips it
# to be descending order. Top actors are 18, 22 and 16.
central_social[order(-central_social$indegree_social),]

# Outdegree: 22, 18 and 19.
central_social[order(-central_social$outdegree_social),]

# In-closeness: 11, 15 and 18.
# NOTE: For some reason, this operation returns strange values;
# a visual inspection of the plot suggests that 11, 15, and 18
# are not central actors at all. This could be a bug.
central_social[order(-central_social$incloseness_social),]

# Out-closeness: 22, 16, and 19
central_social[order(-central_social$outcloseness_social),]
```

```
# Eigenvector: 18, 19, and 16
central_social[order(-central_social$eigen_social),]

# let's make a plot or two with these summary statistics

# To visualize these data, we can create a barplot for each
# centrality measure. In all cases, the y-axis is the value of
# each category and the x-axis is the node number.
barplot(central_social$indegree_social, names.arg=central_social$V.s641_social..name)
barplot(central_social$outdegree_social, names.arg=central_social$V.s641_social..name)
barplot(central_social$incloseness_social, names.arg=central_social$V.s641_social..name)
barplot(central_social$outcloseness_social, names.arg=central_social$V.s641_social..name)
barplot(central_social$betweenness_social, names.arg=central_social$V.s641_social..name)
barplot(central_social$eigen_social, names.arg=central_social$V.s641_social..name)

# Question #2 - What can we say about the social actors if we compare the bar plots?
# Who seems to run the show in sociable affairs? Who seems to bridge sociable conversations?



###
# 4. CORRELATIONS BETWEEN CENTRALITY MEASURES
###

# Now we'll compute correlations betwee the columns to determine
# how closely these measures of centrality are interrelated.

# Generate a table of pairwise correlations.
cor(central_social[,2:7])

# INTERPRETATION:
#
# Indegree and outdegree are very closely correlated (rho = 0.95),
# indicating that social talk with others is reciprocated (i.e.,
# if you talk to others, they tend to talk back to you).
#
# The same is not true of incloseness and outcloseness (rho =
# 0.38), indicating that the closeness calculated from inbound
# paths is not strongly associated with with closeness from
# outbound paths.
#
# In- and out-degree are highly correlated with eigenvector
# centrality, indicating that the students that talk the most to
# others (or, relatedly, are talked to the most by others) are
```

```
# also the ones that are connected to other highly connected
# students -- possibly indicating high density cliques around
# these individuals.
#
# Betweennes shows the highest corelation with outdegree, follwed
# by indegree. In the case of this particular network, it seems
# that the individuals that talk to the most others are the
# likeliest to serve as bridges between the particular cliques
# (see, e.g., 22 in the plot).


###
# 5. REPEAT FOR TASK TALK
###

# Indegree
# We should have 20 entries, indicating 2 isolates.
indegree_task <- degree(s641_task, mode='in')
indegree_task

# Outdegree
outdegree_task <- degree(s641_task, mode='out')
outdegree_task

# In-closeness
incloseness_task <- closeness(s641_task, mode='in')
incloseness_task

# Out-closeness
outcloseness_task <- closeness(s641_task, mode='out')
outcloseness_task

# Betweenness. Note that the closeness measures arent very high
# for node 22, but the betweenness is off the charts.
betweenness_task <- betweenness(s641_task)
betweenness_task

# Eigenvector
s641_task_undirected <- as.undirected(s641_task, mode='collapse')
ev_obj_task <- evcent(s641_task_undirected)
eigen_task <-ev_obj_task$vector
eigen_task
```

```
# Generate a data frame with all centrality values
central_task <- data.frame(V(s641_task)$name, indegree_task, outdegree_task, incloseness_task,
outcloseness_task, betweenness_task, eigen_task)
central_task

# In-degree: 22, 18 and 17
central_task[order(-central_task$indegree_task),]

# Outdegree: 22, 18 and 17
central_task[order(-central_task$outdegree_task),]

# Incloseness: 22, 18 and 17
central_task[order(-central_task$incloseness_task),]

# Outcloseness: 22, 18 and 17
central_task[order(-central_task$outcloseness_task),]

# Eigenvector: 22, 18 and 17
central_task[order(-central_task$eigen_task),]

# Look at barplots
barplot(central_task$indegree_task, names.arg=central_task$V.s641_task..name)
barplot(central_task$outdegree_task, names.arg=central_task$V.s641_task..name)
barplot(central_task$incloseness_task, names.arg=central_task$V.s641_task..name)
barplot(central_task$outcloseness_task, names.arg=central_task$V.s641_task..name)
barplot(central_task$betweenness_task, names.arg=central_task$V.s641_task..name)
barplot(central_task$eigen_task, names.arg=central_task$V.s641_task..name)

# Question #3 - What can we say about the social actors if we compare the bar plots?
# Who seems to run the show in task affairs? Who seems to bridge task conversations?


###
# 6. TASK/SOCIAL CORRELATIONS
###

# Note that in order to do this, we need to either have no missing
# data or use pairwise complete observations.
#
# It would be nice if the centrality functions padded N/A or zero
# data for the isolates, because then the dimensions of the two
# matrices would be compatible. But right now we have 19 nodes for
# social interaction and 20 nodes for task interaction. So first
```

```
# we have to do some hacky R stuff to make them both have 22
# nodes.

# First, we'll extract the node names from the SSL data, using
# levels() because it's a factor and converting it to numbers so
# we can match with the TSL data. Then we'll repeat for TSL.
connectednodes_social = as.numeric(levels(central_social$V.s641_social..name))[central_social$V.
s641_social..name]
connectednodes_task = as.numeric(levels(central_task$V.s641_task..name))[central_task$V.s641_task..
name]

# Check that we did this correctly: SSL should have 19 nodes, and
# TSL should have 20 nodes.
length(connectednodes_social)
length(connectednodes_task)

# Extract matches for each data set, take that subset and use
# columns 2 through 7 to create the correlation matrix. This
# computes the correlations based only on the actors in both
# graphs (18 in total).
cor(central_social[which(connectednodes_social %in% connectednodes_task),2:7], central_task[which
(connectednodes_task %in% connectednodes_social),2:7])



# INTERPRETATION:
#
# eigen_task is correlated with betweenness_social (rho=0.83) and
# outdegree (rho=0.82), possibly because those who are
# important in talk on tasks also serve as bridges for talk on
# social issues and have many outbound ties.
#
# indegree_task and betweenness_social (rho=0.88), and
# outdegree_task and betweenness_social (rho=0.88) are correlated,
# possibly because the number of indegree and outdegree ties a
# node has with respect to task talk, the more they serve as a
# bridge on social talk.
#
# incloseness_task and incloseness_social (rho=0.86) are
# correlated, meaning that those who serve in shortest parths past
# on inbound ties are equivalent for both social talk and task
# talk, which seems to make sense given the betweenness
# correlations with network importance and degree between task and
# social talk more interpretations are possible as well.
```

Question #4 - What can we infer about s641 from these results?
What sort of substantive story can we derive from it?

```
################################################
# LAB 5: Two-Mode Networks and Mobility Models #
################################################


# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.


######################################################################
#
# Lab 5
#
# This lab covers affiliation data. It shows the user how to plot
# affiliation data using the igraph package, how to transform it
# into one mode data and generate centrality measures.

# This lab also introduces additional material on plotting network
# data, with special attention to two-mode networks. It also
# covers using opacity/transparency in network plots, hand-placing
# nodes in a network layout (in case labels overlap), and various
# other considerations that may be of interest when creating
# network visualizations in your publications.
#
# It then covers network mobility. Specifically, it covers how to
# compute transition probability matrices and then instructs the
# user to create visualizations similar to those generated for the
# affiliation data.
#
# We'll work with affiliation data collected by Dan McFarland
# on student extracurricular affiliations (CITE). It's a
# longitudinal data set, with 3 waves - 1996, 1997, 1998.  It
# consists of students (anonymized) and the student organizations
# in which they are members (e.g. National Honor Society,
# wrestling team, cheerleading squad, etc.).
######################################################################

# To-do
# (A) need to add 3 affiliations:
#       1.      "entry" affiliations for students: for any kid younger than 8th grade (for Magnet) or 9th grade (for Rural) in any particular year, this would be their affiliation that year.
#       2.      "leave" affiliations for students: for any kid older than 12th grade in any particular year, this
```

```
would be their affiliation.
#       3.      "non-participant": for any student who is not in a leave or entry position and who has a row sum of 0.
# (B) Using my table of affiliation characteristics, we can omit certain fluff clubs like pep club, NHS, foreign language clubs, etc from the graph as they drive a lot of non-sensical patterns.

# Load the "igraph" library
library(igraph)

# (1) Read in the data files, NA data objects coded as "na"
magact96 = read.delim("http://sna.stanford.edu/sna_R_labs/data/mag_act96.txt", na.strings = "na", check.names = FALSE)
magact97 = read.delim("http://sna.stanford.edu/sna_R_labs/data/mag_act97.txt", na.strings = "na", check.names = FALSE)
magact98 = read.delim("http://sna.stanford.edu/sna_R_labs/data/mag_act98.txt", na.strings = "na", check.names = FALSE)

# Missing data is coded as "na" in this data, which is why we gave
# R the command na.strings = "na". We need to preserve the
# original column names for labeling our visualizations so we use
# the "check.names = FALSE" argument as well. If we needed to
# access our data using the "magact96$variable" syntax, we would
# NOT want to read in our data like this.

# These files consist of four columns of individual-level
# attributes (ID, gender, grade, race), then a bunch of group
# membership dummy variables (coded "1" for membership, "0" for no
# membership).  We need to set aside the first four columns (which
# do not change from year to year).

# take a look at the data as follows (you can edit or "fix" the data
# using this perspective if necessary):
fix(magact96)

#####################################################################
# (2) Create the attribute data. Attributes appear the same
# from 1996-1998
magattrib = magact96[,1:4]

#####################################################################
# (3) Drop columns so we have a square incidence matrix for each
# year
g96 = as.matrix(magact96[,-(1:4)]); row.names(g96) = magact96[,1]
```

```
g97 = as.matrix(magact97[,-(1:4)]); row.names(g97) = magact97[,1]
g98 = as.matrix(magact98[,-(1:4)]); row.names(g98) = magact98[,1]

# For future reference, you can access these via
# data(studentnets.magact96.97.98, package = "NetData")

# Now load in these two-mode graphs into igraph.
i96 <- graph.incidence(g96, mode=c("all") )
i97 <- graph.incidence(g97, mode=c("all") )
i98 <- graph.incidence(g98, mode=c("all") )

#####################################################################
# (3a) Run some plots:
#
# Now, let's plot these graphs. The igraph package has excellent
# plotting functionality that allows you to assign visual
# attributes to igraph objects before you plot. The alternative is
# to pass 20 or so arguments to the plot.igraph() function, which
# gets really messy.

# In past labs, you may have accessed vertex (node attributes via
# this function: get.edge.attribute(krack_full, 'reports_to_tie')

# A "shorthand" to access edge or vertex (node) attributes works
# similarly to R's handling of lists and dataframes (e.g.,
# 'dataframe$variablename' or'list$sublist$object').

# Each node (or "vertex") object is accessible by calling V(g),
# and you can call (or create) a node attribute by using the $
# operator so that you call V(g)$attribute.

# Let's now use this notation to set the vertex color, with
# special attention to making graph objects slightly transparent.
# We'll use the rgb function in R to do this. We specify the
# levels of Red, Green, and Blue and "alpha-channel" (a.k.a.
# opacity) using this syntax: rgb(red, green, blue, alpha). To
# return solid red, one would use this call: rgb(red = 1,green =
# 0,blue = 0, alpha = 1). We will make nodes, edges, and labels
# slightly transparent so that when things overlap it is still
# possible to read them.

# You can read up on the RGB color model at
# http://en.wikipedia.org/wiki/RGB_color_model.
```

```
# Here's how to set the color attribute for a set of nodes in a
# graph object:

V(i96)$color[1:1295] = rgb(red = 1, green = 0, blue = 0, alpha = .5)
V(i96)$color[1296:1386] = rgb(red = 0, green = 1, blue = 0, alpha = .5)

# Notice that we index the V(g)$color object by a seemingly
# arbitrary value, 1295.  This marks the end of the student nodes,
# and 1296 is the first group node. You can view which nodes are
# which by typing V(i96). R prints out a list of all the nodes in
# the graph, and those with an id number can be distinguished from
# group names.

# From here on out, we do not specify "red = ", "green = ", "blue
# = ", and "alpha = ". These are the default arguments (R knows
# the first number corresponds to red, the second to blue, and so
# on).

# Now we'll set some other graph attributes:
V(i96)$label = V(i96)$name
V(i96)$label.color = rgb(0,0,.2,.5)
V(i96)$label.cex = .4
V(i96)$size = 6
V(i96)$frame.color = V(i96)$color

# You can also set edge attributes. Here we'll make the edges
# nearly transparent and slightly yellow because there will be so
# many edges in this graph:

E(i96)$color = rgb(.5,.5,0,.2)

# Now, we'll open a pdf "device" on which to plot. This is just a
# connection to a pdf file. Note that the code below will take a
# minute or two to execute (or longer if you have a pre- Intel
# dual-core processor), because the graph is so large.

pdf("5.1_magact_stdnt_actvts_1996.pdf")
plot(i96, layout=layout.fruchterman.reingold)
dev.off()

# Note that we've used the Fruchterman-Reingold force-directed
# layout algorithm here. Generally speaking, when you have a
```

```
# ton of edges, the Kamada-Kawai layout algorithm works well but,
# it can get really slow for networks with a lot of nodes. Also,
# for larger networks, layout.fruchterman.reingold.grid is faster,
# but can fail to produce a plot with any meaningful pattern if you
# have too many isolates, as is the case here. Experiment for
# yourself.

# Now, if you open the pdf output, you'll notice that you can zoom
# in on any part of the graph ad infinitum without losing any
# resolution. How is that possible in such a small file? It's
# possible because the pdf device output consists of data based on
# vectors: lines, polygons, circles, ellipses, etc., each specified
# by a mathematical formula that your pdf program renders when you
# view it. Regular bitmap or jpeg picture output, on the other
# hand, consists of a pixel-coordinate mapping of the image in
# question, which is why you lose resolution when you zoom in on a
# digital photograph or a plot produced with most other programs.

# This plot is oddly reminiscent of a crescent and star, but
# impossible to read. Part of the problem is the way in which
# layout algorithms deal with isolates. For example,
# layout.fruchterman.reingold will squish all of the connected
# nodes into the center creating a useless "hairball"-like
# visualization. The same applies to
# layout.fruchterman.reingold.grid (it's even worse). And
# layout.kamada.kawai takes exponentially longer to converge (it
# may not ever converge in igraph's implementation of the
# algorithm with isolates).

# Let's remove all of the isolates (the crescent), change a few
# aesthetic features, and replot. First, we'll remove isolates, by
# deleting all nodes with a degree of 0, meaning that they have
# zero edges. Then, we'll suppress labels for students and make
# their nodes smaller and more transparent. Then we'll make the
# edges more narrow more transparent. Then, we'll replot using
# various layout algorithms:

i96 = delete.vertices(i96, V(i96)[ degree(i96)==0 ])
V(i96)$label[1:857] = NA
V(i96)$color[1:857] =  rgb(1,0,0,.1)
V(i96)$size[1:857] = 2

E(i96)$width = .3
```

```
E(i96)$color = rgb(.5,.5,0,.1)

pdf("5.2_magact_stdnt_actvts_1996_layout.kamada.kawai.pdf")
plot(i96, layout=layout.kamada.kawai)
dev.off()

pdf("5.3_magact_stdnt_actvts_1996_layout.fruchterman.reingold.grid.pdf")
plot(i96, layout=layout.fruchterman.reingold.grid)
dev.off()

pdf("5.4_magact_stdnt_actvts_1996_layout.fruchterman.reingold.pdf")
plot(i96, layout=layout.fruchterman.reingold)
dev.off()

# The nice thing about the Fruchterman-Reingold layout in this
# case is that it really emphasizes centrality -- the nodes that
# are most central are nearly always placed in the middle of the
# plot.

# Now repeat for other years (we do not delete the vertices in
# these years.

# 1997
i97 = delete.vertices(i97, V(i97)[ degree(i97)==0 ])
V(i97)$color = rgb(0, 1, 0, .5)

V(i97)$color[1:752] =  rgb(1,0,0,.1)
V(i97)$frame.color = V(i97)$color

V(i97)$size = 6
V(i97)$size[1:752] = 2

V(i97)$label = V(i97)$name
V(i97)$label.color = rgb(0,0,.2,.5)
V(i97)$label.cex = .4
V(i97)$label[1:752] = NA

E(i97)$width = .3
E(i97)$color = rgb(.5,.5,0,.2)

pdf("5.5_magact_stdnt_actvts_1997.pdf")
plot(i97, layout=layout.fruchterman.reingold)
dev.off()
```

```
# 1998
i98 = delete.vertices(i98, V(i98)[ degree(i98)==0 ])
V(i98)$color = rgb(0, 1, 0, .5)
V(i98)$color[1:724] =  rgb(1,0,0,.1)
V(i98)$frame.color = V(i98)$color

V(i98)$size = 6
V(i98)$size[1:724] = 2

V(i98)$label = V(i98)$name
V(i98)$label.color = rgb(0,0,.2,.5)
V(i98)$label.cex = .4
V(i98)$label[1:724] = NA

E(i98)$width = .3
E(i98)$color = rgb(.5,.5,0,.2)

pdf("5.6_magact_stdnt_actvts_1998.pdf")
plot(i98, layout=layout.fruchterman.reingold)
dev.off()

#################################################################
# (4) Now produce single mode co-event matrices for each year. This
# is done using R's matrix algebra commands. You first need to get
# your data in matrix format. We already have a matrix representation
# of our data, but if you did not, a network object can be coerced via
# as.matrix(your-network) if you are using the network or sna
# packages; with the igraph package you would use get.adjacency(your
# network).

# To get the one-mode representation of ties between rows (people in
# our example), multiply the matrix by its transpose. To get the
# one-mode representation of ties between columns (clubs in our
# example), multiply the transpose of the matrix by the matrix.
# Note that you must use the matrix-multiplication operator %*%
# rather than a simple asterisk. The R code is for our data follows:

g96e = t(g96) %*% g96
g97e = t(g97) %*% g97
g98e = t(g98) %*% g98

i96e = graph.adjacency(g96e, mode = "undirected")
```

```
# Now we need to transform the graph so that multiple edges become an
# attribute of each unique edge, accessible via E(g)$weight:

E(i96e)$weight <- count.multiple(i96e)
i96e <- simplify(i96e)

# Now plot the first single mode co-event matrices. Set vertex
# attributes, making sure to make them slightly transparent by
# altering the gamma via the rgb(r,g,b,gamma) function.

# Set vertex attributes
V(i96e)$label = V(i96e)$name
V(i96e)$label.color = rgb(0,0,.2,.8)
V(i96e)$label.cex = .6
V(i96e)$size = 6
V(i96e)$color = rgb(0,0,1,.5)
V(i96e)$frame.color = V(i96e)$color



# We set edge opacity/transparency as a function of how many students
# each group has in common (the weight of the edge that connects the
# two groups).

# In order to do so, we need to transform the edge weights so that
# they are between about .05 and 1, otherwise they will not show up on
# the plot.

# We use the log function + .3 to make sure all transparencies are
# on relatively the same scale, then divide by the maximum edge weight
# to get them on a scale from about .2 and 1.

egalpha = (log(E(i96e)$weight)+.3)/max(log(E(i96e)$weight)+.3)
E(i96e)$color = rgb(.5,.5,0,egalpha)

# For illustrative purposes, let's compare how the Kamada-Kawai and
# Fruchterman-Reingold algorithms render this graph:

pdf("5.7_magact_stdnt_actvts_1996_clubs.pdf")
plot(i96e, main = "layout.kamada.kawai", layout=layout.kamada.kawai)
plot(i96e, main = "layout.fruchterman.reingold", layout=layout.fruchterman.reingold)
dev.off()
```

```
# Be sure to go tot the second page in the pdf to see the FR layout.
# You might like the Kamada-Kawai layout for this graph, because the
# center of the graph is very busy if you use the Fruchterman-Reingold
# layout.

#####################################################################
# (5) and (6) Group Overlap Networks and Plots

# We are also interested in the percent overlap between groups.
# Note that this will be a directed graph, because the percent overlap
# will not be symmetric across groups--for example, it may be that 3/4
# of Spanish NHS members are in NHS, but only 1/8 of NHS members are
# in the Spanish NHS. We'll create this graph for all years in our
# data (though we could do it for one year only).

# First we'll create a percent overlap graph. We start by dividing
# each row by the diagonal (this is really easy in R):

ol96 = g96e/diag(g96e)
ol97 = g97e/diag(g97e)
ol98 = g98e/diag(g98e)

#####################################################################
# (7) Next, we'll sum the matrices and set any NA cells (caused by
# dividing by zero in the step above) to zero:

magall = ol96 + ol97 + ol98
magall[is.na(magall)] = 0

# Note that magall now consists of a percent overlap matrix, but
# because we've summed over 3 years, the maximun is now 3 instead of
# 1.

#####################################################################
# 7 (a) compute average club size, by taking the mean across each
# value in each diagonal
magdiag = apply(cbind(diag(g96e), diag(g97e), diag(g98e)), 1, mean )

#####################################################################
# 7 (a) i. Generate centrality for magall

# First we'll want to make an igraph object:
magallg = graph.adjacency(magall, weighted=T)
```

```
# We need to set weighted=T because otherwise igraph dichotomizes
# edges at 1. Note also that this graph is directed, so we no longer
# tell igraph that the graph is "undirected." The graph is directed
# because the percent overlap will not be symmetric across groups -
# for example, it may be that 3/4 of Spanish NHS members are in NHS,
# but only 1/8 of NHS members are in the Spanish NHS.

# Now we'll calculate some node-level centrality measures:

# Degree
V(magallg)$degree = degree(magallg)

# Betweenness centrality
V(magallg)$btwcnt = betweenness(magallg)

######################################################################
# 7 (b) plot magall, for relationships > 1

# Before we plot this network, we should probably filter some of the
# edges, otherwise our graph will probably be too busy to make sense
# of visually. Take a look at the distribution of connection strength
# by plotting the density of the magall matrix:
pdf("5.8_magact_stdnt_actvts_club_overlap_density.pdf")
plot(density(magall))
dev.off()

# Nearly all of the edge weights are below 1 -- or in other words, the
# percent overlap for most clubs is less than 1/3. Let's filter at 1,
# so that an edge will consists of group overlap of more than 1/3 of
# the group's members in question.
magallgt1 = magall
magallgt1[magallgt1<1] = 0

magallggt1 = graph.adjacency(magallgt1, weighted=T)

# Removes loops:
magallggt1 <- simplify(magallggt1, remove.multiple=FALSE, remove.loops=TRUE)

# Degree
V(magallggt1)$degree = degree(magallggt1)

# Betweenness centrality
```

```
V(magallggt1)$btwcnt = betweenness(magallggt1)

# Before we do anything else, we'll create a custom layout based on
# Fruchterman-Ringold wherein we adjust the coordates by hand using
# the tkplot gui tool to make sure all of the labels are visible. This
# is very useful if you want to create a really sharp-looking network
# visualization for publication.
magallggt1$layout = layout.fruchterman.reingold(magallggt1)
V(magallggt1)$label = V(magallggt1)$name
tkplot(magallggt1)

# Let the plot load, then maximize the window, and select to View ->
# Fit to Screen so that you get maximum resolution for this large
# graph. Now hand-place the nodes, making sure no labels overlap.

# Pay special attention to whether the labels overlap (or might
# overlap if the font was bigger) along the vertical. Save the layout
# coordinates to the graph object:
magallggt1$layout = tkplot.getcoords(1)

# We use "1" here only if this was the first tkplot object you
# called. If you called tkplot a few times, use the last plot object.
# You can tell which object is visible because at the top of the
# tkplot interface, you'll see something like "Graph plot 1".

# Set vertex attributes
V(magallggt1)$label = V(magallggt1)$name
V(magallggt1)$label.color = rgb(0,0,.2,.6)
V(magallggt1)$size = 6
V(magallggt1)$color = rgb(0,0,1,.5)
V(magallggt1)$frame.color = V(magallggt1)$color

# Set edge attributes
E(magallggt1)$arrow.size = .3

# Set edge alpha according to edge weight
egalpha = (E(magallggt1)$weight+.1)/max(E(magallggt1)$weight+.1)
E(magallggt1)$color = rgb(.5,.5,0,egalpha)

# One thing that we can do with this graph is to set label size as a
# function of degree, which adds a "tag-cloud"-like element to the
# visualization:
V(magallggt1)$label.cex = V(magallggt1)$degree/(max(V(magallggt1)$degree)/2)+ .3
```

```
# Note, as presently coded, you must play with the formula above to
# get the ratio of big text to small text just right for other graphs.
# You may want to hand place the nodes as we did earlier using tkplot.

#Now plot it:
pdf("5.9_magact_stdnt_actvts_club_overlap_handplaced_layout.pdf")
plot(magallggt1)
dev.off()

# This plots our network with our custom layout. (Because we specified
# our layout so that it was part of our igraph object as
# "magallggt1$layout," igraph used our layout by default).

##################################################################
# 7 (c) Play around with the visualizations above, save the best

# Variations might include scaling vertex size based on degree,
# scaling edge width or edge transperancy (gamma) based on
# edge weight.

##################################################################
# (8) Play around with cohesion and centrality measurements.
# What can we say about the general comemberships at Magnet
# High over 3 years?

# useful code:

plot(density(degree(i96e)))
plot(density(betweenness(i96e)))

degree.distribution(i96e)

#################################################################
#################################################################
# Part II: Mobility and Careers:
#################################################################

# (1) create a new matrix that multiplies 1996 magnet with 1997
# magnet so you see the number of students moving from 1996
# membership to 1997 memberships.

# Before we actually do this, we need to make sure that the
```

```
# rows and columns for g96 and g97 are the same. We'll
# use the match() function for this:

# First, let's get an idea of how many column-names (activities) and row
# names (student ids) are in common between the two years:

(cnames = intersect( colnames(g96), colnames(g97) ) )
(rnames = intersect( row.names(g96), row.names(g97) ) )

# Great, there are a lot of names in common. Now we
# need to make sure we are only using the rows
# and columns of each matrix that contain entries used in
# both years. We also need to make sure that the columns and
# rows are in the same order.

# In order to accomplish this we are going to exploit R's
# indexing capabilities. We are going to have R "rebuild"
# each matrix according to the order of rnames and cnames.
# We'll use the match() function to accomplish this.
g96matched = g96[ match(rnames, row.names(g96)), match(cnames, colnames(g96)) ]
g97matched = g97[ match(rnames, row.names(g97)), match(cnames, colnames(g97)) ]

# We need to do the same thing for the diagonal of the matrix g96e, which is
# our co-membership/affiliation matrix computed above:
mag96diagmatched = diag( g96e[ match(cnames, colnames(g96e)), match(cnames, colnames(g96e)) ] )

# Now let's check to make sure things worked correctly:
which(row.names(g96matched) != row.names(g97matched))
which(colnames(g96matched) != colnames(g97matched))

# NOW we can multiply to get the transition probability matrix:
mag96_97 = t(g96matched) %*% g97matched

# Now we need to do the same for 97 and 98:
cnames = intersect( colnames(g97), colnames(g98) )
rnames = intersect( row.names(g97), row.names(g98) )
g97matched = g97[ match(rnames, row.names(g97)), match(cnames, colnames(g97)) ]
g98matched = g98[ match(rnames, row.names(g98)), match(cnames, colnames(g98)) ]

mag97_98 = t(g97matched) %*% g98matched
mag97diagmatched = diag( g97e[ match(cnames, colnames(g97e)), match(cnames, colnames(g97e)) ] )
```

```
############################################################
## (2) Create mag96-diag and mag97-diag
## what we'll need to do is to run the same proceedure as above

#mag96diag = diag(g96matched)
#mag97diag = diag(g97matched)

############################################################
# 2 (a) Create magmob96_97 by dividing by mag96diagmatched in
#       order to get a transition probability matrix:
magmob96_97 = mag96_97/mag96diagmatched

############################################################
# 2 a (i) Repeat for mag97 98 using mag97-diag values.
#          Save as magmob97 98.

magmob97_98 = mag97_98/mag97diagmatched

############################################################
# (3) aggregate the transition probability matrix across years

# Now use the match() function to make it so that the
# transition probability matrices have the same row and
# column entries, using the example above as a guide.

# One you have that, you can just add the matrices and
# (scalar) divide by 2.

############################################################
# (4) Now plot as you did with the event-overlap graphs


############################################################
#
# Questions:
#   (1) How does the story differ for the mobility vs the
#       cross-sectional overlaps?
#   (2) What network properties (cohesion or centrality) afford
#       some insight into your claims?
#   (3) From these results, what substantive story can you make
#       about the extracurriculum and it's career structure?
#
############################################################
```

```
##############################################################################
# You may cite these labs as follows: McFarland, Daniel, Solomon Messing,
# Mike Nowak, and Sean Westwood. 2010. "Social Network Analysis
# Labs in R." Stanford University.
##############################################################################


##############################################################################
# LAB 6 - Blockmodeling Lab
# The point of this lab is to introduce students to blockmodeling
# techniques that call for a metric of structural equivalence, a method
# and rationale for the selection of the number of positions, and then
# a means of summary representation (mean cutoff and reduced graph
# presentation). Students will be shown how to identify positions using
# correlation as a metric of structural equivalence (euclidean distance
# is used in earlier lab), and they will be taught how to identify more
# isomorphic notions of role-position using the triad census. Last, the
# lab calls upon the user to compare positional techniques and come up
# with a rationale for why they settle on one over another.
##############################################################################



# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.

###
#1. SETUP
###

library(igraph)
library(sna)
library(triads)
library(psych)
library(nFactors)
library(NetCluster)

###
#2. LOADING AND FORMATTING DATA
###
```

```
data(studentnets.M182, package = "NetData")

# Reduce to non-zero edges and build a graph object
m182_full_nonzero_edges <- subset(m182_full_data_frame, (friend_tie > 0 | social_tie > 0 | task_tie > 0))
head(m182_full_nonzero_edges)

m182_full <- graph.data.frame(m182_full_nonzero_edges)
summary(m182_full)

# Create sub-graphs based on edge attributes
m182_friend <- delete.edges(m182_full, E(m182_full)[E(m182_full)$friend_tie==0])
summary(m182_friend)

m182_social <- delete.edges(m182_full, E(m182_full)[E(m182_full)$social_tie==0])
summary(m182_social)

m182_task <- delete.edges(m182_full, E(m182_full)[E(m182_full)$task_tie==0])
summary(m182_task)

# Look at the plots for each sub-graph
pdf("6.1_m182_studentnet_friend_social_task_plots.pdf", width = 10)
par(mfrow = c(1,3))

friend_layout <- layout.fruchterman.reingold(m182_friend)
plot(m182_friend, layout=friend_layout, main = "friend", edge.arrow.size=.5)

social_layout <- layout.fruchterman.reingold(m182_social)
plot(m182_social, layout=social_layout, main = "social", edge.arrow.size=.5)

task_layout <- layout.fruchterman.reingold(m182_task)
plot(m182_task, layout=task_layout, main = "task", edge.arrow.size=.5)
dev.off()

###
# 3. HIERARCHICAL CLUSTERING ON SOCIAL & TASK TIES
###

# We'll use the "task" and "social" sub-graphs together as the
# basis for our structural equivalence methods. First, we'll use
# the task graph to generate an adjacency matrix.
#
# This matrix represents task interactions directed FROM the
```

```
# row individual TO the column individual.
m182_task_matrix_row_to_col <- get.adjacency(m182_task, attr='task_tie')
m182_task_matrix_row_to_col

# To operate on a binary graph, simply leave off the "attr"
# parameter:
m182_task_matrix_row_to_col_bin <- get.adjacency(m182_task)
m182_task_matrix_row_to_col_bin

# For this lab, we'll use the valued graph. The next step is to
# concatenate it with its transpose in order to capture both
# incoming and outgoing task interactions.
m182_task_matrix_col_to_row <- t(m182_task_matrix_row_to_col)
m182_task_matrix_col_to_row

m182_task_matrix <- rbind(m182_task_matrix_row_to_col, m182_task_matrix_col_to_row)
m182_task_matrix

# Next, we'll use the same procedure to add social-interaction
# information.
m182_social_matrix_row_to_col <- get.adjacency(m182_social, attr='social_tie')
m182_social_matrix_row_to_col

m182_social_matrix_row_to_col_bin <- get.adjacency(m182_social)
m182_social_matrix_row_to_col_bin

m182_social_matrix_col_to_row <- t(m182_social_matrix_row_to_col)
m182_social_matrix_col_to_row

m182_social_matrix <- rbind(m182_social_matrix_row_to_col, m182_social_matrix_col_to_row)
m182_social_matrix

m182_task_social_matrix <- rbind(m182_task_matrix, m182_social_matrix)
m182_task_social_matrix

# Now we have a single 4n x n matrix that represents both in- and
# out-directed task and social communication. From this, we can
# generate an n x n correlation matrix that shows the degree of
# structural equivalence of each actor in the network.
m182_task_social_cors <- cor(m182_task_social_matrix)
m182_task_social_cors

# To use correlation values in hierarchical NetCluster, they must
```

```
# first be coerced into a "dissimilarity structure" using dist().
# We subtract the values from 1 so that they are all greater than
# or equal to 0; thus, highly dissimilar (i.e., negatively
# correlated) actors have higher values.
dissimilarity <- 1 - m182_task_social_cors
m182_task_social_dist <- as.dist(dissimilarity)
m182_task_social_dist

# Note that it is also possible to use dist() directly on the
# matrix. However, since cor() looks at associations between
# columns and dist() looks at associations between rows, it is
# necessary to transpose the matrix first.
#
# A variety of distance metrics are available; Euclidean
# is the default.
#m182_task_social_dist <- dist(t(m182_task_social_matrix))
#m182_task_social_dist

# hclust() performs a hierarchical agglomerative NetCluster
# operation based on the values in the dissimilarity matrix
# yielded by as.dist() above. The standard visualization is a
# dendrogram. By default, hclust() agglomerates clusters via a
# "complete linkakage" algorithm, determining cluster proximity
# by looking at the distance of the two points across clusters
# that are farthest away from one another. This can be changed via
# the "method" parameter.

pdf("6.2_m182_studentnet_social_hclust.pdf")
m182_task_social_hclust <- hclust(m182_task_social_dist)
plot(m182_task_social_hclust)
dev.off()

# cutree() allows us to use the output of hclust() to set
# different numbers of clusters and assign vertices to clusters
# as appropriate. For example:
cutree(m182_task_social_hclust, k=2)

# Now we'll try to figure out the number of clusters that best
# describes the underlying data. To do this, we'll loop through
# all of the possible numbers of clusters (1 through n, where n is
# the number of actors in the network). For each solution
# corresponding to a given number of clusters, we'll use cutree()
# to assign the vertices to their respective clusters
```

```
# corresponding to that solution.
#
# From this, we can generate a matrix of within- and between-
# cluster correlations. Thus, when there is one cluster for each
# vertex in the network, the cell values will be identical to the
# observed correlation matrix, and when there is one cluster for
# the whole network, the values will all be equal to the average
# correlation across the observed matrix.
#
# We can then correlate each by-cluster matrix with the observed
# correlation matrix to see how well the by-cluster matrix fits
# the data. We'll store the correlation for each number of
# clusters in a vector, which we can then plot.

# First, we initialize a vector for storing the correlations and
# set a variable for our number of vertices.
clustered_observed_cors = vector()
num_vertices = length(V(m182_task))

# Next, we loop through the different possible cluster
# configurations, produce matrices of within- and between-
# cluster correlations, and correlate these by-cluster matrices
# with the observed correlation matrix.

pdf("6.3_m182_studentnet_task_social_clustered_observed_corrs.pdf")
clustered_observed_cors <-clustConfigurations(num_vertices,m182_task_social_hclust,
m182_task_social_cors)
clustered_observed_cors
plot(clustered_observed_cors$correlations)
dev.off()

clustered_observed_cors$correlations
# From a visual inspection of the correlation matrix, we can
# decide on the proper number of clusters in this network.
# For this network, we'll use 4. (Note that the 1-cluster
# solution doesn't appear on the plot because its correlation
# with the observed correlation matrix is undefined.)
num_clusters = 4
clusters <- cutree(m182_task_social_hclust, k = num_clusters)
clusters

cluster_cor_mat <- clusterCorr(m182_task_social_cors,
                               clusters)
```

```
cluster_cor_mat

# Let's look at the correlation between this cluster configuration
# and the observed correlation matrix. This should match the
# corresponding value from clustered_observed_cors above.
gcor(cluster_cor_mat, m182_task_social_cors)



######################
# Questions:
# (1) What rationale do you have for selecting the number of
# clusters / positions that you do?
######################



### NOTE ON DEDUCTIVE CLUSTERING

# It's pretty straightforward, using the code above, to explore
# your own deductive NetCluster. Simply supply your own cluster
# vector, where the elements in the vector are in the same order
# as the vertices in the matrix, and the values represent the
# cluster to which each vertex belongs.
#
# For example, if you believed that actors 2, 7, and 8 formed one
# group, actor 16 former another group, and everyone else formed
# a third group, you could represent this as follows:
deductive_clusters = c(1, 2, 1, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1,
                       1, 3)

# You could then examine the fitness of this cluster configuration
# as follows:
deductive_cluster_cor_mat <- generate_cluster_cor_mat(
  m182_task_social_cors,
  deductive_clusters)
deductive_cluster_cor_mat
gcor(deductive_cluster_cor_mat, m182_task_social_cors)

### END NOTE ON DEDUCTIVE CLUSTERING

# Now we'll use the 4-cluster solution to generate blockmodels,
# using the raw tie data from the underlying task and social
# networks.
```

```
# Task valued
task_mean <- mean(m182_task_matrix_row_to_col)
task_mean

task_valued_blockmodel <- blockmodel(m182_task_matrix_row_to_col, clusters)
task_valued_blockmodel

# Task binary
task_density <- graph.density(m182_task)
task_density

task_binary_blockmodel <- blockmodel(m182_task_matrix_row_to_col_bin, clusters)
task_binary_blockmodel


# Social valued
social_mean <- mean(m182_social_matrix_row_to_col)
social_mean

social_valued_blockmodel <- blockmodel(m182_social_matrix_row_to_col, clusters)
social_valued_blockmodel

# Social binary
social_density <- graph.density(m182_social)
social_density

social_binary_blockmodel <- blockmodel(m182_social_matrix_row_to_col_bin, clusters)
social_binary_blockmodel

# We can also permute the network to examine the within- and
# between-cluster correlations.

cluster_cor_mat_per <- permute_matrix(clusters, cluster_cor_mat)
cluster_cor_mat_per


######################
# Questions:
# (2) What is the story you get from viewing these clusters,
# and their within and between cluster densities on task and
# social interaction? What can you say about M182 from this?
######################
```

```
###
# 4. HIERARCHICAL CLUSTERING ON TRIAD CENSUS
###

# Another way to think about roles within a network is by looking
# at the triads that each actor belongs to. We can then use
# correlations between triad-type memberships to identify people
# with similar roles regardless of the specific people with whom
# they interact.

# First, we'll generate an individual-level triad census of the
# network using triadcensus() from the triads package.
task_triads <- triadcensus(m182_task)
task_triads

# Next, we'll generate a matrix of correlations between actors
# in the network based on their similarity in triad-type
# membership. Note that the cor() function in R operates on
# columns, not rows, so in order to get correlations between
# the actors in the network we have to transpose it.
m182_task_triad_cors <- cor(t(task_triads))
m182_task_triad_cors

# As above, we can use the correlation matrix to generate a
# dissimilarity structure, which we can then hierarchically
# cluster into groups of similar people.
dissimilarity <- 1 - m182_task_triad_cors
m182_task_triad_dist <- as.dist(dissimilarity)
m182_task_triad_dist

m182_task_triad_hclust <- hclust(m182_task_triad_dist)

pdf("6.4_m182_studentnet_task_triad_hclust.pdf")
plot(m182_task_triad_hclust)
dev.off()

# As above, we'll loop through each possible cluster solution
# and see how well they match the observed matrix of triad-type
# correlations.
clustered_observed_cors = vector()
num_vertices = length(V(m182_task))
```

```
pdf("6.5_m182_studentnet_task_hclust_triad_corrs.pdf")
clustered_observed_cors <-clustConfigurations(num_vertices,m182_task_triad_hclust,
m182_task_triad_cors)
dev.off()

clustered_observed_cors


# From a visual inspection of the data, we'll use a 3-cluster
# solution (though a case could also be made for using 5.)
num_clusters = 3
clusters <- cutree(m182_task_triad_hclust, k = num_clusters)
clusters

cluster_cor_mat <- clusterCorr (m182_task_triad_cors,
            clusters)

cluster_cor_mat
gcor(cluster_cor_mat, m182_task_triad_cors)


# As before, we can use these clusters to run a blockmodel
# analysis using the underlying tie data from the task network.

# Task valued
task_mean <- mean(m182_task_matrix_row_to_col)
task_mean

task_valued_blockmodel <- blockmodel(m182_task_matrix_row_to_col, clusters)
task_valued_blockmodel

# Task binary
task_density <- graph.density(m182_task)
task_density

task_binary_blockmodel <- blockmodel(m182_task_matrix_row_to_col_bin, clusters)
task_binary_blockmodel


# Finally, we can try to get a sense of what our different
# clusters represent by generating a cluster-by-triad-type matrix.
# This is an m x n matrix, where m is the number of clusters and n
```

```
# is the 36 possible triad types. Each cell is the average
# number of the given triad type for each individual in the
# cluster.
cluster_triad_mat <- matrix(nrow=max(clusters), ncol=ncol(task_triads))
for (i in 1:max(clusters)) {
	for (j in 1:ncol(task_triads)) {
		cluster_triad_mat[i,j] <- mean(task_triads[which(clusters==i),j])
	}
}

cluster_triad_mat



######################
# Questions:
# (3) What does clustering of the triadic census afford us?
# What roles do you see? Redo the initial blockmodel analysis
# without social interaction (only task) and then compare to
# this solution. Do they differ?
#
# Extra credit: Try running the triad census on task AND
# social interaction separately and then correlating persons.
# What result do you get? Is it different from our initial
# blockmodel result? Show your code.
######################




###
# 5. FACTOR ANALYSIS
###

# Note that although we are conducting a principal components
# analysis (PCA), which is technically not exactly the same as
# factor analysis, we will use the term "factor" to describe the
# individual components in our PCA.

# PCA is often used in network analysis as a form of detecting
# individuals global positioning. We say "global" because these
# clusters aren't defined on local cohesion but from the overall
# pattern of ties individuals have with all others (structural
# equivalence). Identifying the first two largest components that
# organize the variance in tie patterns is one way of doing this.
```

```
# We'll analyze the 4n x n matrix generated above.

# First, we want to determine the ideal number of components
# (factors) to extract. We'll do this by examining the eigenvalues
# in a scree plot and examining how each number of factors stacks
# up to a few proposed non-graphical solutions to selecting the
# optimal number of components, available via the nFactors
# package.
ev <- eigen(cor(m182_task_social_matrix)) # get eigenvalues
ap <- parallel(subject=nrow(m182_task_social_matrix),
            var=ncol(m182_task_social_matrix),
            rep=100,cent=.05)
nS <- nScree(ev$values, ap$eigen$qevpea)

pdf("6.6_m182_studentnet_task_social_pca_scree.pdf")
plotnScree(nS)

# To draw a line across the graph where eigenvalues are = 1,
# use the following code:
plotnScree(nS)
abline(h=1)
dev.off()



# For more information on this procedure, please see
# the references provided in the parallel() documentation
# (type "?parallel" in the R command line with the package
# loaded).

# Now we'll run a principal components analysis on the matrix,
# using the number of factors determined above (note this may not
# be the same number as you get):
pca_m182_task_social = principal(m182_task_social_matrix, nfactors=5, rotate="varimax")

# Let's take a look at the results in the R terminal:
pca_m182_task_social

# You can see the standardized loadings for each factor for each
# node. Note that R sometimes puts the factors in a funky order
# (e.g. RC1, RC2, RC5, RC4, RC3) but all of the factors are there.
# You can see that the SS loadings, proportion of variance
# explained and cumulative variance explained is provided below. A
```

```
# Chi Square test of the factors and various other statistics are
# provided below.

# Note that the eigenvalues can be accessed via the following
# command:
pca_m182_task_social$values

# Now we will use the factor loadings to cluster and compare that
# to our other NetCluster techniques, using dendrograms.

# Take the distance based on Euclidian Distance
m182_task_factor_dist = dist(pca_m182_task_social$loadings)

# And cluster
m182_task_factor_hclust <- hclust(m182_task_factor_dist)

pdf("6.7_m182_studentnet_task_social_pca_hclust.pdf")
plot(m182_task_factor_hclust)
dev.off()

# And compare to NetCluster based on correlations and triads:
pdf("6.8_m182_task_cluster_by_correlation_PCA_Triads.pdf")
par(mfrow = c(1,3))
plot(m182_task_social_hclust, main = "Correlation")
plot(m182_task_factor_hclust, main = "PCA")
plot(m182_task_triad_hclust, main = "Triads")
dev.off()




######################
# Questions:
# (4) How do the results across blockmodel techniques differ?
# Why might you use one over the other? Why might you want to
# run more than one in your analyses?
######################
```

```
##############################################
# Lab 7: PEER INFLUENCE & QAP REGRESSION LAB #
##############################################

# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.

#############################################################################
#
# Lab 7
#
# Part I - Develop peer influence models that meet the high standards of
# publication in todays top journals (i.e., addressing autocorrelation
# and issues of causality).
#
# Part II - Use QAP regression to predict increases in social interaction
# and task interaction for two classrooms. Basically run dyadic level
# regressions suitable for continuous variables and count data (what
# ERGM- Exponential Random Graph's cannot do).
#
#############################################################################



#############################################################################
# PART I -- PEER INFLUENCE
#############################################################################
#
# This lab examines peer effects in classroom data. It first introduces
# the reader to the necessary data processing and manipulation, then basic
# visualization related to peer-effects. It then introduces the Linear Network
# Autocorrelation Model (lnam), which can be used to better model the
# dependencies in network data (well, better than an OLS model anyway).
#
# The next section introduces the reader to the concept of matching--treating
# the explanatory variable of interest as an experimental "treatment"
# and then matching on other covariates. Matching allows the application
# of straightforward analytical techniques used to analyze experiments,
# namely comparison of means. Code for bootstrapping is introduced to allow for
# non-parametric estimation of the standard errors for the mean. Code is also provided
# to produce a dotplot showing the means and a (bootstrapped) 95% CI.
#
# The data was collected by Dan McFarland (McFarland 2001). The key measures of interest
# for this example are the time 1 and 2 measures of how much each student liked
```

```
# the subject, and the friend network at time 1.
#
# The codebook is available here: http://stanford.edu/~messing/codebook.v12.txt
##############################################################################

#Clear Variables
rm(list=ls(all=TRUE))
gc()

# Install and load necessary packages:
install.packages("reshape", dependencies = TRUE)
library(reshape)
library(igraph)

data(studentnets.peerinfl, package="NetData")

##############################################################################################
# Now we need to reshape our attitudes data according to semester. The data is
# currently in "long" format, so that student ids and measures are repeated on
# multiple lines and "semester" is a single variable. We need
# to transform the data so that measures are repeated on a single
# line, or "wide" format. We'll use the excellent "reshape" package
# to accomplish this. Take a look at the documentation using ?reshape

?reshape
attitudesw = reshape( attitudes, idvar="std_id",
            timevar="sem_id", direction="wide" )

# Notice that all semester 1 variables now feature a ".1" postscript, while all
# semester 2 variables feature a ".2" postscript

# We'll first visualize the data. We want to know whether the change in
# a student's appreciation of a subject (t_2 - t_1) changes in the direction of his
# friends' appreciation at t_1.

# So, we'll first take the difference of a few dependent variables
# for which we might expect peer influence (e.g. "sub" (liking of subject),
# "cmt" (liking of classmates), and "tch" (liking of teacher)),
# and then take the mean value for an individual's friends' values at t_1.

# First create the delta variables:
attitudesw$deltasub = attitudesw$sub.2 - attitudesw$sub.1
attitudesw$deltatch = attitudesw$tch.2 - attitudesw$tch.1
attitudesw$deltacmt = attitudesw$cmt.2 - attitudesw$cmt.1
```

```
# Next we'll create the mean of friends' sub.1.
# We'll use the adjacency matrix, multiply that by the attitudesw$sub.1
# (element-wise, not matrix multiplication), and then take the mean of each row.

sem1graph = graph.data.frame(d = sem1[1:2], directed=TRUE)
#sem1graph = network(x = sem1[1:2], directed=TRUE)
#sem2graph = graph.data.frame(d = sem2, directed=TRUE)

# But first, we'll need to clean the data to make sure the rows line up!

# let's check to see whether the edge data has the same cases as the attitudes data
which(!(V(sem1graph)$name %in% as.character(attitudesw$std_id)))
which(!(as.character(attitudesw$std_id) %in% V(sem1graph)$name ))

# They are not the same... we'll need to address this below.

# Now let's get the matrix representation of the network:
sem1matrix = get.adjacency(sem1graph, binary=T)
# This is often referred to as "W" in the literature

# When you have a large square matrix like this, you can get a good idea of
# it's density/sparsity using the image function.
image(sem1matrix)

# It's also generally good to know the degree distribution of any network you
# are handling:

plot(density(degree(sem1graph)))

# Looks like degree might be distributed exponentially. We can check as follows:
simexpdist100 = rexp(n=length(V(sem1graph)), rate = .100)
simexpdist125 = rexp(n=length(V(sem1graph)), rate = .125)
simexpdist150 = rexp(n=length(V(sem1graph)), rate = .150)
lines(density(simexpdist100), col="red")
lines(density(simexpdist125), col="blue")
lines(density(simexpdist150), col="green")

# It's not a precise fit, but it might be a good approximation.

# Let's reorder the attitudes data to make sure it's in the same order as
# sem1matrix
attitudesw = attitudesw[match(row.names(sem1matrix), attitudesw$std_id),]
```

```
# Make sure it worked (this should return 0 if so):
which(row.names(sem1matrix)!=as.character(attitudesw$std_id))
which(colnames(sem1matrix)!=as.character(attitudesw$std_id))

# Let's also make sure that igraph read the graph in correctly:
# (note that igraph can behave strangely if you pass it a
#  parameter like "directed = FALSE" for a directed network like this one).

sem1$alter_id[sem1$std_id == 149824]
V(sem1graph)[0]
V(sem1graph)[unique(neighbors(sem1graph, v=0, mode = 1))]

# looks good.

# Now we can compute the mean of sub.1 for a student's friends,
# by element-wise multiplying sub.1 by the matrix and then taking the
# mean of each non-zero cell in each row:

# Let's first test this out with a simple matrix to make sure we know what
# we are doing:
(M = matrix(c(1,0,1,0,1,0,1,0,1), nrow=3, ncol=3))

(R = c(1,2,3))

R * M

# This is multiplying the first row by 1, the second row by 2, and so on.
# Instead we want:

t(R * t(M))

# This is multiplying the first column by 1, the second column by 2, and so on.
# Recall that we will be analyzing this data so that rows are cases, so
# this is what we want if we are going to calculate sub.1 for each
# student's friends' sub.1. (The first option would return the
# student's sub.1 for each non-zero row, not his/her friends' sub.1).

sem1Wxsub1 = t(attitudesw$sub.1 * t(sem1matrix))
sem1Wxtch1 = t(attitudesw$tch.1 * t(sem1matrix))
sem1Wxcmt1 = t(attitudesw$cmt.1 * t(sem1matrix))

# Now we'll take the mean of cells not equal to zero:
attitudesw$frsub1mean = numeric(length(attitudesw$std_id))
attitudesw$frtch1mean = numeric(length(attitudesw$std_id))
```

```
attitudesw$frcmt1mean = numeric(length(attitudesw$std_id))
for(i in 1:length(attitudesw$std_id)){
	attitudesw$frsub1mean[i] = mean(sem1Wxsub1[i,sem1Wxsub1[i,]!=0 ], na.rm=T)
	attitudesw$frtch1mean[i] = mean(sem1Wxtch1[i,sem1Wxtch1[i,]!=0], na.rm=T)
	attitudesw$frcmt1mean[i] = mean(sem1Wxcmt1[i,sem1Wxcmt1[i,]!=0], na.rm=T)
}

# Now let's plot the data and look at the relationship:
plot(attitudesw$frsub1mean, jitter(attitudesw$deltasub))
plot(attitudesw$frtch1mean, jitter(attitudesw$deltatch))
plot(attitudesw$frcmt1mean, jitter(attitudesw$deltacmt))

# Looks noisy, but there might be a relationship for "sub".

#############################################################################
# An alternative way to compute this is iteratively. The advantage
# to the following approach is that if you are working with a large
# data set, you do not need to load an adjacency matrix into
# memory, you can keep things in edgelist format, which is MUCH
# more efficient.

attitudesw$mfrsub.1 = numeric(length(attitudesw$sub.1))
attitudesw$mfrtch.1 = numeric(length(attitudesw$sub.1))
attitudesw$mfrcmt.1 = numeric(length(attitudesw$sub.1))

# If you thought the number of alters who like the class mattered
# more than the mean, you might uncomment the code for the nfrsubgt3
# variable and incorporate it into your analysis (two occurrences):
#attitudesw$nfrsubgt3 = numeric(length(attitudesw$sub.1))

for(i in 1:length(attitudesw$std_id)){
	# first get alters of student i:
	altrs = sem1$alter_id[ sem1$std_id == attitudesw$std_id[i] ]

	# then get alters' attitudes
	altatts = attitudesw$sub.1[ attitudesw$std_id %in% altrs ]

	# now count how many friends like the class more than "3"
	 attitudesw$nfrsubgt3[i] = length(which(altatts > 3))

	# then take the mean, ignoring NAs:
	attitudesw$mfrsub.1[i] = mean(altatts, na.rm = TRUE)

	# Note that this can all be done in one line of code:
```

```
      # attitudesw$mfrsub.1[i] = mean(attitudesw$sub.1[ attitudesw$std_id %in% sem1$alter_id[sem1
$std_id %in% attitudesw$std_id[i]]], na.rm=TRUE )
       attitudesw$mfrtch.1[i] = mean(attitudesw$tch.1[ attitudesw$std_id %in% sem1$alter_id[sem1
$std_id %in% attitudesw$std_id[i]]], na.rm=TRUE )
       attitudesw$mfrcmt.1[i] = mean(attitudesw$cmt.1[ attitudesw$std_id %in% sem1$alter_id[sem1
$std_id %in% attitudesw$std_id[i]]], na.rm=TRUE )
}
# if you are going to run this with a lot of data, you may wish to include
# the following lines inside your for-loop:
# gc()
# print(i)

# this will run the garbage collector, gc(), which helps R manage memory
# and print(i) will let you know your progress as R chugs away.

# The plot is exactly the same:
plot(attitudesw$mfrsub.1, jitter(attitudesw$deltasub))

# And the correlation should be 1
cor(attitudesw$mfrsub.1,attitudesw$frsub1mean, use= "complete")

# The plots suggest that there might be a relationship for "sub", or
# how much each student likes the subject.

# Let's cheat a little bit and run linear models predicting
# change in each of our variables based on mean friend's values at t=1.
# The data violates many of the assumptions of OLS regression so the
# estimates are not particularly meaningful, other than to let us know
# that we may be on the right path.

summary(lm(deltasub~mfrsub.1, data=attitudesw))
summary(lm(deltatch~mfrtch.1, data=attitudesw))
summary(lm(deltacmt~mfrcmt.1, data=attitudesw))

# Significance is always encouraging, even though we are only predicting a tiny
# fraction of the variance (R^2 = .026)

# We'll cheat a little more and remove everyone whose attitudes did not change:
summary(lm(deltasub~mfrsub.1, data=attitudesw, subset = deltasub!=0 ))
summary(lm(deltatch~mfrtch.1, data=attitudesw, subset = deltasub!=0 ))
summary(lm(deltacmt~mfrcmt.1, data=attitudesw, subset = deltasub!=0 ))

# Very nice, for "sub" the effect grows in strength, signficance,
# and R^2 goes up to .067.
```

```
# Let's also take a look at the plot:
pdf("7.1_Peer_effect_on_opinion_of_subject.pdf")
plot(   x = attitudesw$mfrsub.1,
            y = jitter(attitudesw$deltasub),
            main = "Peer effects on opinion of subject",
            xlab = "mean friends' opinion",
            ylab = expression(Delta~"opinion from "~t[1]*~to~t[2])
)

# We can draw a regression line on the plot based on the regression
# we estimated above:
abline(lm(deltasub~mfrsub.1, data=attitudesw ))
dev.off()

# We've got some evidence of a peer effect on how much a student
# likes the subject. Our evidence is based on temporal precendence:
# your friends opinion of the subject predicts how much
# your opinion the subject changes from t_1 to t_2.

#############################################################################
# Now let's run some "real" models.
# We'll first estimate the model Y_2 = Y_1 + W*Y_alter, using a network
# autocorrelation model. Read up on it here:

# Roger Th. A. J. Leenders, Modeling social influence through network
# autocorrelation: constructing the weight matrix, Social Networks, Volume 24, Issue 1, January 2002,
Pages 21-47, ISSN 0378-8733, DOI: 10.1016/S0378-8733(01)00049-1.
# (http://www.sciencedirect.com/science/article/B6VD1-44SKCC2-1/2/
cae2d6b4cf4c1c21f4f870fd2d58b5cc)

# This model arguably takes into accounts the correlation between friends in
# the disturbances (error term). Yet there are problems with this kind of model.
# See:

# Eric J. Neuman, Mark S. Mizruchi, Structure and bias in the network autocorrelation
# model, Social Networks, In Press, Corrected Proof, Available online 31 May 2010, ISSN 0378-8733,
DOI: 10.1016/j.socnet.2010.04.003.
# (http://www.sciencedirect.com/science/article/B6VD1-506H0SN-
1/2/73a16c627271d29d9283b6d69b873a07)

# With that in mind, let's proceed:
library(sna)
library(numDeriv)
```

```
# We'll use the mfrsub.1 variable to approximate the W*Y_alter term. (If
# we actually use the matrix, we cannot estimate the model--there
# would be the same number of variables as cases).

# Let's remove the NA values from the attitudes measures
# (otherwise lnam will crash). We'll remove rows with NAs for
# sub.1, sub.2, or mfrsub.1. Note that we could remove any row with an
# NA value with the syntax: na.omit(attitudesw), but if there are
# rows that contain our variables of interest but are NA for some other
# variable, na.omit would also drop these rows and we would lose valuable
# data.

atts = attitudesw[!is.na(attitudesw$sub.2),]
atts = atts[!is.na(atts$sub.1),]
atts = atts[!is.na(atts$mfrsub.1),]
W = sem1matrix

# Now we'll make sure the rows and columns in W are in the same
# order as atts:
W = W[match(atts$std_id,row.names(W)), match(atts$std_id,colnames(W))]
# Let's make sure (this will return 0 if we did it right):
which(rownames(W) != atts$std_id)

# Now we'll estimate the peer influence model using lnam, which stands for
# Linear Network Autocorrelation Model.
pim1<-lnam(atts$sub.2,cbind(atts$sub.1, atts$mfrsub.1), W2 = W)
summary(pim1)

# This shows a pretty strong peer effect on subjects at time2. Notice that
# the adjusted R^2 is .41. This is not good news. Even after controlling
# for t1 values of attitude for the subject, we are predicting less than
# half of the variance in t_2 values. That means that there is a good
# chance of omitted variable bias.

# Below is an attempt to run the model with the actual WYalt as the X matrix.
# For the reasons stated above, it doesn't work. It is included in case the
# reader wants to uncomment this code and see for him/herself.

#WYalt = sem1Wxsub1
#WYalt = WYalt[match(atts$std_id,row.names(WYalt)), match(atts$std_id,colnames(WYalt))]
#image(WYalt)
#
##pim2<-lnam(y = atts$sub.2, x = WYalt, W2 = W)
```

```
#pim2<-lnam(y = atts$sub.2, x = WYalt)
#summary(pim2)


##############################################################################
# Matching
#
# One method that is less vulnerable to some problems related to regression is
# matching. In matching, one divides the key explanatory variable into discrete
# "treatments" and then matches participants so that they are as similar on the
# remaining set of covariates as possible. A simple difference in means/medians/
# quantiles can be used. From there we can use a simple t-test to estimate the
# "treatment" effect without having to worry about multicolinearity.
# If the data violates the normality assumption (as will be the
# case here), non-parametric methods such as a permutation test or bootstrapped
# standard errors can be used. This approach also also violates less statistical
# assumptions and therefore should be expected to be less biased than running a
# simple OLS model.

# For the original article on matching, see:
# Donald B. Rubin, Matching to Remove Bias in Observational Studies
# Biometrics, Vol. 29, No. 1 (Mar., 1973), pp. 159-183
# Stable URL: http://www.jstor.org/stable/2529684

# Also worthy of note is the Rubin Causal Model, which uses matching.
# Read up on it here: http://en.wikipedia.org/wiki/Rubin_Causal_Model

# However, matching is not a cure-all and is obviously sensitive to the variables
# on which one matches. For an excellent article on this, see:

# Jeffrey A. Smith, Petra E. Todd, Does matching overcome LaLonde's critique of
# nonexperimental estimators?, Journal of Econometrics, Volume 125, Issues 1-2,
# Experimental and non-experimental evaluation of economic policy and models,
# March-April 2005, Pages 305-353.
# (http://www.sciencedirect.com/science/article/B6VC0-4CXTY59-
1/2/06728bd79899fd9a5b814dfea9fd1560)

# A good introduction to matching using the MatchIt package can be found courtesy
# of Gary King here: http://gking.harvard.edu/matchit/docs/matchit.pdf

library(MatchIt)

# First, we dichotomize the independent variable of interest, mean friend opinion regarding
# subject: mfrsub.1
```

```
atts$mftreat = ifelse(atts$mfrsub.1 > 3, 1,0)

atts = atts[c("mftreat", "deltasub", "mfrsub.1","egrd.1", "sub.1", "tot.1", "frn.1", "cmt.1", "prev.1")]

# Before we do any matching, let's take a look at our new treatment variable.

# We'll make this into a factor, which will help interpret our results. A factor
# is a special type of object in R that is used for qualitative (nominal or ordinal)
# data/variables. A factor is also generally more efficient with respect to memory,
# because it stores an integer value corresponding to a label (e.g. 1 for "low" and 2
# for "high") rather than storing the entire string.

# Factors have an order, which you can set by ordering the labels in ascending order
# when you create the factor, or by using the reorder() function. We'll talk about this
# more in the section on dotplots below.
atts$mftreatf = factor(atts$mftreat, labels = c("low", "high"))

# Let's compute the means:
tapply(X = atts$deltasub, INDEX = list(atts$mftreatf), FUN = mean)

# Note, the tapply function is useful if you have more than one factor for which
# you want to calcuate some function over. For example, if there were another
# factor, say exptreat, we could get the mean for each cell using this syntax:
# tapply(X = atts$deltasub, INDEX = list(atts$mftreat, atts$exptreat), FUN = mean)

# A t-test is not totally kosher here because the DV is ordinal, and so it violates
# assumptions behind the t-test.

# But let's take a look anyway:
t.test(deltasub~mftreatf, data=atts)

# A better alternative is the Wilcoxon test (a.k.a. Mann-Whitney test),
# suitable for ordinal variables.
wilcox.test(x=atts$deltasub[atts$mftreatf=="low"],
            y=atts$deltasub[atts$mftreatf=="high"],
            conf.int = TRUE)

# Now let's match and redo the analyses.

# We normally would assign each subject/case to treatment or control groups based on
# whatever corresponds more closely to a treatment or a control. In this case, it
# is debatable whether having friends who like the class is more of a "treatment"
# than having friends who do not like the class. However, there are 246 students
# whose friends do not like the class versus 99 whose friends do like the class,
```

```
# so we can say that having friends who like the class is certainly more unusual
# than the opposite. Accordingly, we'll conceptualize having friends who like the
# class as the treatment.

# In the actual matching proceedure, what we'll do is try to find individuals in
# the "control" condition (having friends who do not like the subject) who look
# most similar to each treated individual based on a variety of variables. Ideally,
# we will achieve "balance" on the covariates between those in the treatment condition,
# and those we select from the control condition, so that the individuals are as similar
# as possible.

# Here are the variables we'll use (all are at time 1):

# sub.1 - how much the student likes the subject
# egrd.1 - expected grade
# tot.1 - how much the student likes the way the course is taught
# frn.1 - how interesting the subject finds his/her friends
# cmt.1 - how interesting the student finds his/her classmates
# prev.1 - has the student had the teacher previously)

# We'll use "nearest neighbor" matching, which uses a distance measure to
# select the individual in the "control" group that is closest to each
# individual in the "treatment" group. The default distance measure is the
# logistic regression propensity score.

# The codebook for this data is available here: http://stanford.edu/~messing/codebook.v12.txt

m.out = matchit(mftreat ~ egrd.1 + sub.1 + tot.1 + frn.1 + cmt.1 + prev.1,
            data = atts, method = "nearest")

summary(m.out)
plot(m.out)

# Now let's assess the extent to which our matching algorithm effectively matched each
# treatment subject to a control subject, e.g., the extent to which we achieved balance.
# The plot function displays Q-Q plots of the control (X-axis) versus treatment groups
# (Y-axis) for the original sample and the matched sample. The Q-Q plots indicate perfect
# balance on the covariate in question if all dots are perfectly aligned on the
# 45 degree line. The further the deviation from this line, the worse the samples are
# balanced on this variable.

# Based on the plots, matching appears to have improved balance on our covariates a
# little bit, but not perfectly.
```

```
# Ideally we want to experiment with other covariates and try to find the combination
# that best and most parsimoniously captures the key phenomena we want to control for.

matchatts = match.data(m.out)

# We'll make our treatment variable into a factor:
matchatts$mftreatf = factor(matchatts$mftreat, labels = c("low", "high"))

# Let's compute the means:
tapply(X = matchatts$deltasub, INDEX = list(matchatts$mftreatf), FUN = mean)

t.test(deltasub~mftreatf, data=matchatts)

wilcox.test(x=matchatts$deltasub[matchatts$mftreatf=="low"],
            y=matchatts$deltasub[matchatts$mftreatf=="high"],
            conf.int = TRUE)

# We can also perform a full permuation test. A permutation test is an exact test of
# whether it is possible to reject the null hypothesis that two distributions are
# the same. It works by first calulating the mean difference, which fuctions as the
# test statistic. The difference in sample means is calculated and recorded for every
# possible way of dividing these pooled values into two groups of size n_A and n_B
# for every permutation of the group labels A and B. The set of these calculated
# differences is the exact distribution of possible differences under the null
# hypothesis that group label does not matter. This test fuctions like a t.test but
# does not rely on any distributional assumptions about the data.

# For additional information, see:
# http://en.wikipedia.org/wiki/Resampling_%28statistics%29#Permutation_tests

library(coin)
independence_test(deltasub ~ mftreatf, data = matchatts,
            distribution = exact())

# Another alternative is to look at bootstrapped estimates of the mean and standard
# error. This is often an attractive way to estimate statistical significance
# because our data violates the distributional assumptions involved in estimating
# standard errors in this way (namely, that our dependent variable resembles anything
# like a normal or t distribution--it cannot because it is ordinal with only
# 7 levels). Because boostrapping relies on resampling instead of distributional
# assumptions to estimate variance, it is more robust.

# There are packages available to generate bootstrapped estimates, but seeing the code
# is a valuable way to get a sense of exactly how bootstrapping works. Here is the code
```

```
# for a bootstrapping function designed to estimate the mean and standard error:

b.mean <- function(data, nsim) {
	# Create a list object to store the sets of resampled data (in this case nsim = 1000).
	resamples = list()

	# Create a vector of the same length to store the resampled means
	r.mean = numeric(nsim)

	# Generate a sample with replacement
	for(i in 1:nsim){
		# generates a random sample of our data with replacement
		resamples[[i]] = sample(x=data, size=length(data), replace=T)

		# Now calcuate the mean for this iteration
		r.mean[i] = mean(resamples[[i]], na.rm=T)
	}

	# Calculate the mean of the mean of the simulated estimates above:
	mean.adj = mean(r.mean, na.rm=T)

	# Calculate how this differs from the arithmatic mean estimate of the original
	# sample:
	bias = mean(data, na.rm=T) - mean.adj

	# Generate the standard error of the estimate
	std.err <- sqrt(var(r.mean))

	# Return results
	return( data.frame(mean = mean(data, na.rm=T), #the mean estimate
				mean.adj = mean.adj, # the adjusted estimate based on simulations
				bias = bias, # the mean minus the adjusted estimate
				std.err=std.err # the standard error of the estimate (based on simulations)
		)
	)
}

# Before we use it on our data, let's make sure it works. We will simulate some
# data for which we know the parameters and then estimate the parameters using
# the bootstrap:

simdata = rnorm(n = 1000, mean = 5, sd = 1)
b.mean(simdata, nsim=1000)
```

```
# Calculate the theoretical mean and standard error
mean(simdata)
(se = sd(simdata)/sqrt(length(simdata)-1))
# here we use the formula SE = SD/sqrt(N - 1)

# We have demonstrated that our bootstraping function is a good estimator
# for "perfect" data that meets the assumptions behind traditional
# statistical tests. Now let's move on to our data.

# For those with friends with on average postive attitudes:
b.mean(data=matchatts$deltasub[matchatts$mftreatf=="high"],
            nsim=1000)

# For those with friends with on average negative attitudes:
b.mean(data=matchatts$deltasub[matchatts$mftreatf=="low"],
            nsim=1000)

# Here's how to do it using the boot package, which is faster and more
# extensible than the R-code above:

library(boot)

# You have to write a special function for the boot package, which
# can be confusing at first, but is useful for more complicated
# types of bootstrapping. The boot package is also orders of magnitude
# faster than doing things in R, which is useful if you start to do
# more complicated bootstrapping or use a higher number of simulations.

# First, write a basic function that will return the estimate in question
# for x data using d cases:

samplemean <- function(x, d) {
      return(mean(x[d]))
}

(bootlow = boot(data = matchatts$deltasub[matchatts$mftreatf=="low"],
                           statistic = samplemean,
                           R = 1000))

(boothigh =boot(data = matchatts$deltasub[matchatts$mftreatf=="high"],
                           statistic = samplemean,
                           R = 1000))

# Note that estimates of the standard errors will be slightly different
```

```
# each time due to randomness involved in estimation. As you might expect,
# as the number of simulations increases, variance will decrease.

# Based on our bootstrapped estimates and standard error calculations,
# it's clear that the two means are quite different. So, based on our
# matched samples, it seems there is a significant peer effect.



#############################################################################
# Let's try full matching so that we are not throwing away any of our data:
m.out = matchit(mftreat ~ egrd.1 + sub.1 + tot.1 + frn.1 + cmt.1 + prev.1,
            data = atts, method = "full")
matchatts = match.data(m.out)

# We'll make this into a factor, which will help inturpret our results:
matchatts$mftreatf = factor(matchatts$mftreat, labels = c("low", "high"))

# Note that we'll now have to use the weights that the output provided because
# we told it to use the full sample.

# Let's compute the means:
tapply(X = matchatts$deltasub,
            INDEX = list(matchatts$mftreatf),
            FUN = weighted.mean, weights = matchatts$weights )

# There is no straightforward way to compute a t-test in R with weighted data.

# We can however compute weighted standard errors and a corresponding
# 95% CI:

# Recall that a 95% CI of an estimate is calculated by taking the estimate
# +/- the standard error * 1.96. Actually, +/- 1.96 is just an estimate of the
# the 0.05 and 0.95 quantiles of a statistical distribution. We'll use
# qt(quantile, N-1) in this case.
library(Hmisc)
meanlow = wtd.mean(matchatts$deltasub[matchatts$mftreatf=="low"],
            weights = matchatts$weights[matchatts$mftreatf=="low"])
varlow = wtd.var(matchatts$deltasub[matchatts$mftreatf=="low"],
            weights = matchatts$weights[matchatts$mftreatf=="low"])
selow = sqrt(varlow)/sqrt(sum(matchatts$weights[matchatts$mftreatf=="low"]))
meanlow + selow * qt(.025, ( sum(matchatts$weights[matchatts$mftreatf=="low"]) - 1))
meanlow + selow * qt(.975, ( sum(matchatts$weights[matchatts$mftreatf=="low"]) - 1))

meanhigh = wtd.mean(matchatts$deltasub[matchatts$mftreatf=="high"],
```

```
        weights = matchatts$weights[matchatts$mftreatf=="high"])
varhigh = wtd.var(matchatts$deltasub[matchatts$mftreatf=="high"],
        weights = matchatts$weights[matchatts$mftreatf=="high"])
sehigh = sqrt(varhigh)/sqrt(sum(matchatts$weights[matchatts$mftreatf=="high"]))
meanhigh + sehigh * qt(.025, ( sum(matchatts$weights[matchatts$mftreatf=="high"]) - 1))
meanhigh + sehigh * qt(.975, ( sum(matchatts$weights[matchatts$mftreatf=="high"]) - 1))

# Here's how to do it with the boot package:
sample.wtd.mean <- function(x, d, wts) {
    return(weighted.mean(x[d], w = wts))
}

(bootlow = boot(data = matchatts$deltasub[matchatts$mftreatf=="low"],
                    wts = matchatts$weights[matchatts$mftreatf=="low"],
                    statistic = sample.wtd.mean,
                    R = 1000))
(boothigh =boot(data = matchatts$deltasub[matchatts$mftreatf=="high"],
                    wts = matchatts$weights[matchatts$mftreatf=="high"],
                    statistic = sample.wtd.mean,
                    R = 1000))

# We can make a nice plot of the results with the lattice package:
library(lattice)

# Dot plots:
visual = list()
visual$var = c("Friends do not like subj.", "Friends like subj.")
visual$M = c(bootlow$t0, boothigh$t0)
visual$se = c(sd(bootlow$t), sd(boothigh$t))
visual$N = c(length(matchatts$weights[matchatts$mftreatf=="high"]),
        length(matchatts$weights[matchatts$mftreatf=="low"]))
visual$lo = visual$M - visual$se
visual$up = visual$M + visual$se
visual = as.data.frame(visual)

#print it out:
pdf(file="7.2_dotplot_matchoutput_bootstrapped_SE.pdf", width = 6, height = 3)
dotplot( reorder(var,M) ~ M,
        data = visual,
        main="Effect of friends' attitude on opinion change,\nmatched samples with bootstrapped SE",
        panel = function (x, y, subscripts, groups, ...) {
            panel.dotplot(x, y)
            panel.segments(x0=visual$lo[subscripts],y0=as.numeric(y),
                    x1=visual$up[subscripts],y1=as.numeric(y),
```

```
                        lty = 1 )
        },
        xlab=expression(Delta~"opinion from "~t[1]*~to~t[2]),
        xlim= c(-.5, .2)
)
dev.off()

# Very nice!

##############################################
# QUESTION #1 - What do these results show?
# What assumptions are being made?
# What might lead you to question the results? What would improve them?
##############################################

#########################################################################
# Extra-credit:
#
# Use the McFarland dataset to acquire other individual level variables
# and develop better matching. Then follow this lab and explore the
# variety of outcomes suitable for illustrating peer effects in
# classrooms (e.g., PSAT scores, engagement, conflict, etc). Results
# are likely suitable for an A-journal publication.
#
# Again, the codebook is available here: http://stanford.edu/~messing/codebook.v12.txt
#
#########################################################################



#########################################################################
# PART II -- QAP REGRESSION
#########################################################################
#
#      Here we want to compare different classrooms and discern what leads
# participants to become more or less sociable and academically engaged
# with one another.
#
#      Class m173 is an accelerated math class of all 10th graders taught
# by a rigid teacher who used teacher-centered instructional formats.
# Class m 182 is a regular math class of 10th and 11th grade students
# taught by a casual teacher who used student-centered instructional formats.
#
#########################################################################
```

```
# igraph and sna don't mix well.  Before we load sna, we will detach igraph.
detach(package:igraph)

# Load the "sna" library
library(sna)

#Clear Variables
rm(list=ls(all=TRUE))
gc()

#(2) QAP Regressions for m l73

# Note that each matrix must be the same size (n x n). You'll want to make
# sure all input and output matrices are the same size. Predictor matrices
# thus are not individual level variables; they are differences, matches,
# etc. You'd use a matrix of differences in of quantitative (node level)
# variables, and matches in the case of categorical/dichotomous (node
# level) variables. There's a really easy way to do it in R
# (say your node-level variables are x and y)
# x <- seq(1,4)
# y <- seq(2,5)
# outer(x,y,FUN="-") # find the difference between x and y
# outer(x,y,FUN="==") # produce 0/1 similairty matrix
# For this lab we will use matrices that were previously generated in UCINet

# Loading predictor matrices.  Each matrix is a n x n matrix
# data is saved in a CSV format.  You can open the files in Excel
# to see the structure.

data(studentnets.mrqap173, package="NetData")

# Look at what we loaded via
ls()

# We need the data in matrix format
# predictor matrices
m173_sem1_SSL <- as.matrix(m173_sem1_SSL)
m173_sem1_TSL <- as.matrix(m173_sem1_TSL)
m173_sem1_FRN <- as.matrix(m173_sem1_FRN)
m173_sem1_SEAT <- as.matrix(m173_sem1_SEAT)
m173_sem1_RCE <- as.matrix(m173_sem1_RCE)
m173_sem1_GND <- as.matrix(m173_sem1_GND)
```

```
# Load response matrices
m173_sem2_SSL <- as.matrix(m173_sem2_SSL)
m173_sem2_TSL <- as.matrix(m173_sem2_TSL)

# In order to run the QAP regression we must create an array of matrices
# containing all the predcitor matrices.  We are, in effect, creating a
# 3-d matrix (predcitor x n x n).

# Important: All matrices must be the same size!

response_matrices <- array(NA, c(6, length(m173_sem1_SSL[1,]),length(m173_sem1_SSL[1,])))
response_matrices[1,,] <- m173_sem1_SSL
response_matrices[2,,] <- m173_sem1_TSL
response_matrices[3,,] <- m173_sem1_FRN
response_matrices[4,,] <- m173_sem1_SEAT
response_matrices[5,,] <- m173_sem1_RCE
response_matrices[6,,] <- m173_sem1_GND

################################
#(2a) SSL2 <- SSL1 + TSL1 + FRN1 + SEAT1 + RCE + GND
################################

# Fit a netlm model by using netlm, the response matrix and the array of predictor matrices
# This may take a LONG time.
nl<-netlm(m173_sem2_SSL,response_matrices)

# Make the model easier to read by adding lables for each predictor matrix.
nlLabeled <- list()
nlLabeled <- summary(nl)
# Labels are provided in the same order as they were assigned in the response_matrices array
nlLabeled$names <- c("Intercept", "Social normalized and labeled (SSL1)", "Task normalized and labeled
(TSL1)", "Friends 1=friend, 2=best friend(FRN1)", "Seat in first semester (Seat1)","Race (RCE)","Gender
(GND)")

# Round the ocefficients to two decimals
nlLabeled$coefficients = round(nlLabeled$coefficients, 2)
nlLabeled

################################
#(2b) TSL2 <- TSL1 + SSL1 + FRN1 + SEAT1 + RCE + GND
################################

# Fit a netlm model by using netlm, the response matrix and the array of predictor matrices
nl<-netlm(m173_sem2_TSL,response_matrices)
```

```
#make the model easier to read
nlLabeled <- list()
nlLabeled <- summary(nl)
nlLabeled$names <- c("Intercept", "Social normalized and labeled (SSL1)", "Task normalized and labeled (TSL1)", "Friends 1=friend, 2=best friend(FRN1)", "Seat in first semester (Seat1)","Race (RCE)","Gender (GND)")

nlLabeled$coefficients = round(nlLabeled$coefficients, 2)
nlLabeled

##############################
#(3) QAP Regressions for m 182
##############################

# Repeat for class m 182

# Clear Variables
rm(list=ls(all=TRUE))
gc()

data(studentnets.mrqap182, package = "NetData")

# Look at what we loaded via
ls()

# Again, we need the data in matrix format
# predictor matrices
m182_sem1_SSL <- as.matrix(m182_sem1_SSL)
m182_sem1_TSL <- as.matrix(m182_sem1_TSL)
m182_sem1_FRN <- as.matrix(m182_sem1_FRN)
m182_sem1_SEAT <- as.matrix(m182_sem1_SEAT)
m182_sem1_RCE <- as.matrix(m182_sem1_RCE)
m182_sem1_GND <- as.matrix(m182_sem1_GND)

#response matrices
m182_sem2_SSL <- as.matrix(m182_sem2_SSL)
m182_sem2_TSL <- as.matrix(m182_sem2_TSL)

#This class will require you to make multiple extraction operations.
#A student exits the class at the end of first semester and another
#enters at the start of second semester. These students must be removed
#before you can conduct QAP regression (you need the same row-column orderings).
#Extract actor 15 for TSL and SSL of second semester (# 15 <- new student).
```

```
response_matrices <- array(NA, c(6, length(m182_sem1_SSL[1,]),length(m182_sem1_SSL[1,])))
response_matrices[1,,] <- m182_sem1_SSL
response_matrices[2,,] <- m182_sem1_TSL
response_matrices[3,,] <- m182_sem1_FRN
response_matrices[4,,] <- m182_sem1_SEAT
response_matrices[5,,] <- m182_sem1_RCE
response_matrices[6,,] <- m182_sem1_GND

################################
#(3a) SSL2 <- SSL1 + TSL1 + FRN1 + SEAT1 + RCE + GND
################################

#Fit a netlm model
nl<-netlm(m182_sem2_SSL,response_matrices)

#make the model easier to read
nlLabeled <- list()
nlLabeled <- summary(nl)
nlLabeled$names <- c("Intercept", "Social normalized and labeled (SSL1)", "Task normalized and labeled
(TSL1)", "Friends 1=friend, 2=best friend(FRN1)", "Seat in first semester (Seat1)","Race (RCE)","Gender
(GND)")

nlLabeled$coefficients = round(nlLabeled$coefficients, 2)
nlLabeled

################################
#(3b) TSL2 <- SSL1 + TSL1 + FRN1 + SEAT1 + RCE + GND
################################

#Fit a netlm model
nl<-netlm(m182_sem2_TSL,response_matrices)

#make the model easier to read
nlLabeled <- list()
nlLabeled <- summary(nl)
nlLabeled$names <- c("Intercept", "Social normalized and labeled (SSL1)", "Task normalized and labeled
(TSL1)", "Friends 1=friend, 2=best friend(FRN1)", "Seat in first semester (Seat1)","Race (RCE)","Gender
(GND)")

nlLabeled$coefficients = round(nlLabeled$coefficients, 2)
nlLabeled

#Report your results. Describe what they mean? Repeat for ETSL of second semester.
```

```
##########################################################################
# QUESTION #2 - Compare your results for m 173 and m 182.
# Do the classes have different results?
# If not, why not?
# If so, why so?
# What sort of story can you derive about the change in
# task and social interactions within these classrooms?
#
# Remember you are predicting changes in relations of sociable or task
# interaction using other types of relations (i.e., friendship,
# proximate seating, same race, etc).

##########################################################################
#
# Extra-credit - run the QAP regression part of the lab on the
# entire McFarland dataset of classrooms, and perform meta-analyses
# on the results using multi-level modeling - then presto -
# you will have results on academic and social relations in classrooms
# which is suitable for publication in an A-journal.
#
##########################################################################
```

```
##############################################################################
# Lab 8: ERGM LAB
##############################################################################
# Lab goals:
# 1) To create ERGM models
# 2) To compare ERGM models
# 3) Consider ERGM performance complications
##############################################################################


# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.

#outline
#1) ERGM creation
#2) MCMC diagnostics
#3) ERGM model simulation
#4) Model comparison
#5) ERGM performance improvements


# load the "ergm" library
library(ergm)

## Load the data:
data(studentnets.ergm173, package = "NetData")

# The IDs in the data correspond to IDs in the complete dataset.
# To execute the ERGM, R requires continuous integer IDs: [1:n],
# where n is the total number of nodes in the ERGM. So, create
# node IDs acceptable to R and map these to the edges.

# Create 22 unique and sequenced IDs
id <- seq(1,22,1)

# Join these IDs to the nodes data (cbind = column bind), and
# reassign this object to 'nodes'
nodes<-cbind(id, nodes)

# Check the new nodes data to see what we've got.  Notice that we
# now have integer-increasing IDs as a vector in our data frame.
nodes
```

```
# Merge the new IDs from nodes with the ego_id and alter_id values
# in edges. Between merge steps, rename variables to maintain
# consistency.  Note that you should check each data step using
# earlier syntax.  Note that R requires the same ordering for node
# and edge-level data by ego_id.  The following sequence preserves
# the edgelist ordering, rendering it consistent with the
# node ordering.
edges2<-merge(nodes[,1:2], edges, by.x = "std_id", by.y="alter_id")

# Note that we lose some observations here.  This is because the
# alter_id values do not exist in the node list.  Search will
# indicate that these IDs are also not in the set of ego_id values.
names(edges2)[1]<-"alter_id"

# just assigning new names to first column.
names(edges2)[2]<-"alter_R_id"
edges3<- merge(nodes[,1:2], edges2, by.x = "std_id", by.y="ego_id")

# shows that we merged new alter id that reflects
# integer id which R requires.
names(edges3)[1]<-"ego_id"
names(edges3)[2]<-"ego_R_id"
edges3
# The edges3 dataset now contains integer-increasing IDs sorted by
# ego_R_id. For our work, we will use the ego_R_id and alter_R_id
# values, but we retain the std_id values for reference.

# Specify the network we'll call net - where dyads
# are the unit of analysis...
net<-network(edges3[,c("ego_R_id", "alter_R_id")])

# Assign edge-level attributes - dyad attributes
set.edge.attribute(net, "ego_R_id", edges[,2])
set.edge.attribute(net, "alter_R_id", edges[,4])

# Assign node-level attributes to actors in "net"
net %v% "gender" <- nodes[,3]
net %v% "grade" <- nodes[,4]
net %v% "race" <- nodes[,5]
net %v% "pci" <- nodes[,6]

# Review some summary information regarding the network to make
```

```
# sure we have #specified things correctly.
summary(net)

# Let's take a look at the network.
pdf("8.1_lab8_network.pdf")
plot(net)
dev.off()

# Let's execute a model in which we attempt to explain semester 2
# friendship selections exclusively with node-level
# characteristics.
m1<-ergm(net ~ edges + mutual + nodematch("gender") + absdiff
      ("pci"),burnin=15000,MCMCsamplesize=30000,verbose=FALSE)

# The ERGM runs by an MCMC process with multiple starts, and this
# helps you see if the model is converging.  If the estimated
# coefficient values were to change dramatically, it might be a
# sign of a poorly specified model).  You should see the
# log-likelihood increase with each iteration.

# You will see a loop warning.  You can ignore this for now.

# Before trying to interpret the data, it is a good idea to check
# the MCMC process
pdf("8.2_lab8_mcmc_m1.pdf")
mcmc.diagnostics(m1)
dev.off()

# You will see several plots and output.  The plots to the right
# should look approximately normal. The output tells us three
# things of interest:

# 1) The accuracy of the model (r)
# 2) If we used a sufficiently large burn-in
# 3) If we used a sufficiently large sample in the simulation

# In our case the samples might be too small.  This doesn't mean
# the results of the ERGM results are wrong, but we should take
# care in specifying the sample size.

# Let's look at the summary of the results.  We could create a new
# object that is the summary score info, but here we'll just send
# it to the screen.
```

```
# Let's assess the model
summary(m1)

# show the exp() for the ERGM coefficients
lapply(m1[1],exp)

# The first section gives the model (formula) estimated in the
# ERGM. Here we said that the network was a function of edges,
# mutual ties and matching with respect gender.  Another way
# to think about this, is that we're generating random networks
# that match the observed network with respect to the number of
# edges, the number of mutual dyads, the number of ties within
# between race and within/between gender.

# The second section tells how many iterations were done.  The
# MCMC sample size is the number of random networks generated in
# the production of the estimate.

# The next section gives the coefficients, their SEs and pValues.
# These are on a log-odds scale, so we interpret them like logit
# coefficients.

# An edges value of -2.314 means that the odds of seeing a tie on
# any dyad are exp(-2.314) ~= 0.098, which could be thought of as
# density net of the other factors in the model. If you only
# have 'edges' in the model, then exp(b1) should be very close to
# the observed density. Edges are equivalent to a model intercept
# -- while possible, I can't imagine why one would estimate a
# model without an edges parameter.

# A mutual value of 2.412 means that reciprocity is more common
# than expected by chance (positive and significant), and here we
# see that exp(2.412)=11.15, so it's much more likely than chance
# -- we are 11 times as likely to see a tie from ij if ji than if
# j did not nominate i.

# We are exp(1.574e-02)=1.015 times more likely to nominate within
# gender than across gender.

# The final section refers to overall model fit and MCMC
# diagnostic statistics (AIC, BIC).
```

```
# Let's now create a couple of additional networks so that we can
# add earlier friendships and seating proximity to our model.
# We'll do this 2 different ways.  For seating, we'll create an
# entirely new network.  For friend_sem1, we'll assign additional
# attributes to the original network.  These are interchangeable.
seat <- net

# Assign an edge-level attribute of 'seat' to capture the network
# of seating we create a proximity network via seating location...
set.edge.attribute(seat, "seat_net", edges3[,7])

# Assign an edge-level attribute of 'net' to capture sem1
# friendships.
set.edge.attribute(net, "friend1", edges3[,5])

# Note: thus far, we've treated gender as a homogenous matching
# parameter.  We can alternatively allow this effect to vary
# across grades.  Do this by adding a 'diff=TRUE' option for the
# nodematch term.  Many terms have options that change their
# effect, so look at the help files to clarify.

# Create variables to represent sem1 mutuality and transitivity
# Create a new network based on the sem1 friendships.  Use the
# network commands to convert this to a matrix.
test<-edges["sem1_friend">=1,]

test2<-merge(nodes[,1:2], test, by.x = "std_id", by.y="alter_id")
names(test2)[1]<-"alter_id"
names(test2)[2]<-"alter_R_id"
test3<- merge(nodes[,1:2], test2, by.x = "std_id", by.y="ego_id")
names(test3)[1]<-"ego_id"
names(test3)[2]<-"ego_R_id"
net1<-network(test3[,c("ego_R_id", "alter_R_id")])

A<-as.matrix(net1)
B<-t(as.matrix(net1)) #B = A transpose
mut_mat <- A + B
lag_mut<-as.network(mut_mat) # relies on dichotomous
                    # interpretation of edges

# Calculate sem1 transitivity using A matrix from above
# This is highly colienar with our response variable and will
# cause the ERGM to fail. For a different network, you would use
```

```
# the code below to calculate semester 1 transitvity:
# sqA<-A%*%A #matrix multiplication
# sem2_trans<-sqA*A #element-wise multiplication
# sem2_trans_net <- as.network(sem2_trans)

# Create another model that uses the sem1 mutuality
m2<-ergm(net ~ edges + mutual + nodematch("gender") +
      nodematch("race")  + edgecov(lag_mut),burnin=20000,
      MCMCsamplesize=70000,verbose=FALSE,seed=25,
      calc.mcmc.se = FALSE,maxit=6)

pdf("8.3_lab8_mcmc_m2.pdf")
mcmc.diagnostics(m2)
dev.off()

summary(m2)
# We might get a warning here.  This means that R was unable to
# compute standard errors for all predictors.  This could be due
# to a number of causes for the purpose of this example we ignore
# the waring and move on, but in your work you will want to check
# your data for potential problems

# Now let's look at goodness of fit.  In addition to the standard
# GOF statistics, we can use the simulation features of the
# program to see if our models match reality.  Since the models
# are effectively proposals about what is driving the observed
# network, we can ‘back predict from the model to produce a set
# of random networks that are draws from the distribution of
# networks implied by the model.  We can then compare the
# predicted model to the observed model for features not built
# into the model.  So, for example, if the only features
# generating the global network in reality are mixing by grade and
# race, then we should get matching levels of transitivity,
# geodesic distances and so forth with the predicted model.  The
# tools for doing this are (a) to simulate from the model and (b)
# to use the built in GOF functions.

# (a) simulating networks from an estimated model
# The higher the value of nsim the longer this will take
m2.sim<-simulate(m2,nsim=100);

simnet1<-m2.sim$networks[[1]]
summary(simnet1)
```

```
pdf("8.4_lab8_m2_simulation.pdf")
plot(m2.sim$networks[[1]],vertex.col="WHITE")
dev.off()

# Note the resulting net looks a lot like what we have estimated.
# You could easily simulate, say, 1000 nets from your model and
# the write a loop that pulls statistics of interest out of each
# one (like centralization or some such), to compare against your
# observed network.

#This is, essentially, what the built-in GOF models do….

# (b) Generating GOF fits from an estimated model
# the built in goodness-of-fit routine is also very useful.
m2.gof <- gof(m2~idegree)
pdf("8.5_lab8_m2_gof.pdf")
plot(m2.gof)
dev.off()

# This figure plots the distribution of simulated popularity (in-
# degree) as box-plots, with the observed values overlain.  Here
# we see a pretty-good fit, particularly in the middle and tail
# regions. Recall that in model 5, popularity is *not* one of the
# parameters in the model, so this suggests that with the features
# we do include, we can account for the observed degree
# distribution.

# There are also a number of advanced options for running ERGM
# models designed to (a) allow one to specify structural
# parameters of interest, (b) evaluate the convergence of the
# MCMC, and (c) test for degenerate models (models that look
# like they fit, but that actually predict an odd portion of the
# graph sample space).


# LAB QUESTIONS:

# 1. On your own, using these variables and the 'summary(<model
# name>)' command, explore the model that you believe to be the best
# one.  Explain its strengths relative to the other models and the
# logic that suggests this answer to you.

# 2. Why don't we use the node-level variable 'grade' for any of
```

```
# the models?  Using the syntax above as a guide, include 'grade'
# in a variant of m1, m1.2, and report the results from R.

# 3. Describe what we did to calculate the mutuality and
# transitivity scores.

# 4. Describe each of the command terms: edgecov, mutual,
# edges, nodematch, and absdiff.
# NOTE: the command '

help("ergm-terms")

#will be very useful here.

##################################
#improving ergm performance
##################################

# ergm is slow, but modern computers can help a lot.
# an ergm model tries to compute the same general result multiple
# times we can use many threads to harness the power of multicore
# processors we do this with the parallel arguement in ergm

#####WARNING######
# if you are not using a multicore processor this will slow down
# your analysis for most new computers you should use parallel=4.

# let's run the model 4 again with four threads.

m2_fast<-ergm(net ~ edges + mutual + nodematch("gender") +
       nodematch("race")  + edgecov(lag_mut),burnin=20000,
       MCMCsamplesize=70000,verbose=FALSE,seed=25,
       calc.mcmc.se = FALSE,maxit=6,parallel=4)

# this takes a while to run...

summary(m2)
```

```
#####################################################################
# LAB 9: Converting igraph to SoNIA with R
#####################################################################


# NOTE: if you have trouble because some packages are not installed,
# see lab 1 for instructions on how to install all necessary packages.
# Also see Lab 1 for prior functions.


#############################################################
# SoNIA visualizes networks based on vertex and edge (arc) data.
# A number of variables commonly used in iGraph graph objects
# directly map to SoNIA counterparts.  At minimum, SoNIA requires:
# Vertex ids and an edgelsit of directed interactions.
#
# Note: For clarity, the iGraph terms Vetex and Edge are used in
# this lab in place of SoNIA's node and arc.
#############################################################



###
# 1. SET UP SESSION
###

library('igraph')
library("igraphtosonia")

#####################################################################
# SoNIA requirements
# For vertex (node) entires, we must supply:
#
#
# NodeId - must be an integer. values can be used more than once
#       (to specify changes in a node's attributes over time) BUT
#       MUST FORM A CONTINUOUS SEQUENCE. (if you try to leave out
#       numbers it will throw an error, as this would mess up the
#       matrix references.
#
# Additionally, the following vetex attirbutes are supported by
# the igraphToSoNIA package:
```

```
#
# Label - text to be displayed as the vertex label.
# ColorName - text specifying a color for the vertex, one of:
#       Black DarkGray LightGray White Cyan Green Magenta Orange
#       Pink Red Yellow Blue
# NodeShape - text specifying the shape for the node, current
#       can only be "ellipse" or "rect"
# NodeSize - positive real number specifying the size of the
#       vertex in pixels
# StartTime - real value specifying the start time for the node
# EndTime - real value specifying the end time for the node.
#
#
#
# For edge (arc) entires, we must supply:
#
# FromId - integer (or string, if aplpha id is used)
#       indicating the ego node must match with a vetext (node) id
# ToId - integer indicating the alter node must match
#       with a vetext (node) id
#
# Additionally, the following edge attirbutes are supported by
# the igraphToSoNIA package:
#
# ArcWeight - real value indicating the strength of the relation
# ArcWidth - real value indicating how wide to draw the arrow
# ColorName text specifying a color for the edge, one of:
#       Black DarkGray LightGray White Cyan Green Magenta Orange
#       Pink Red Yellow Blue.
# StartTime - real value indicating
#       the edge's start
# EndTime - real value indicating the edge's termination
#####################################################################

# The first step is to create an iGraph graph object.  In practice,
# this code is most useful if you already have an iGraph graph you wish
# to visualize with SoNIA

edgelist <- read.csv('http://sna.stanford.edu/sna_R_labs/data/edges.csv',header=T)
attributes <- read.csv("http://sna.stanford.edu/sna_R_labs/data/vertices.csv",header=T)
graph <- graph.data.frame(d= edgelist, directed=T, vertices = attributes)

#####################################################################
```

```
# Important
# If some of the optional attirbues are not present in your
# data comment out the entry to remove the attribute from your
# export and to instruct SoNIA to use the default value
#####################################################################

# Set required vertex attribute

# set node label
V(graph)$label <- V(graph)$name

# Set optional vertex attributes.  If we will not use a vertex
# attribute in SoNIA, we will set the attribute to NA

# set vertex color (named colors per the list above)
V(graph)$frame.color = c("red","blue","gray")[V(graph)$polticalparty]

# set vertex shape (circle and rectangle only)
V(graph)$shape = c("circle","rectangle")[V(graph)$gender]
V(graph)$frame.shape = V(graph)$shape

# set vertex size
V(graph)$vertex.size =10

# if we do not have individual start and end times for the nodes,
# do not set a value for these nodes by commenting out the following lines
# values of 0 and 30 will display all vertices at all times in SoNIA
# set start time for the node
V(graph)$start.time <-1

# set end time for the node
V(graph)$end.time <-6



# The fromId and toId will be set by igraph to sequential values
# by default. If V(graph)$label is used, then the ids will be strings
# corresponding to the value set.

# set optional edge attributes

# set start time for the edge
E(graph)$start.time <-E(graph)$start
```

```
# set end time for the edge
E(graph)$end.time <-E(graph)$start

# set edge weight
E(graph)$weight <- runif(ecount(graph))

# set the edge arrow width

E(graph)$arrow.size=1.6

# set edge color (named colors per the list above)
E(graph)$color = "blue"

# create the .son output
write.graph.to.sonia(graph,"export.son")

# when you load the file in SoNIA you MUST rescale the layout.  It will
# look as if nothing was loaded, but it is just a bug in SoNIA. "Rescaling
# layout to fit window" will fix the issue.
```